

```
// ===== 文件：game.js（程序起始 / 第 1 个源文件） =====
// game.js —— 抖音小游戏入口（合成能量 / Merge Energy）
// 纯 Canvas 渲染，无外部素材，规避版权风险；广告位以占位形式接入，需填入真实 adUnitId。

const Logic = require('./src/logic.js');
const config = require('./config.js');
const HMAC = require('./src/hmac.js'); // 纯 JS HMAC-SHA256（抖音运行时无 Node crypto）
const Ladder = require('./src/ladder.js'); // 天梯（异步匹配）前端客户端 + Canvas UI
const Room = require('./src/room.js').createRoomClient(); // 房间对战前端客户端 + Canvas UI（Phase 2）

const isTT = typeof tt !== 'undefined' && !!tt.createCanvas;
let canvas = null;
let ctx = null;

let W = 375;
let H = 667;
let dpr = 1;
if (isTT) {
  try {
    const sys = tt.getSystemInfoSync();
    W = sys.windowWidth;
    H = sys.windowHeight;
    dpr = sys.pixelRatio || 1;
  } catch (e) {
    // 使用默认值
  }
  canvas = tt.createCanvas();
  canvas.width = Math.floor(W * dpr);
  canvas.height = Math.floor(H * dpr);
  ctx = canvas.getContext('2d');
  ctx.scale(dpr, dpr);
}

const SIZE = Logic.SIZE;
const PAD = 12;
const boardSize = Math.min(W, H) * 0.88;
const cell = (boardSize - PAD * (SIZE + 1)) / SIZE;
const boardX = (W - boardSize) / 2;
const boardY = (H - boardSize) / 2 + 24;

// 天梯 UI 布局（供 src/ladder.js 绘制使用，单一数据源）
const L = { W, H, PAD, boardX, boardY, boardSize, cell };

const COLORS = {
  2: '#eee4da', 4: '#ede0c8', 8: '#f2b179', 16: '#f59563', 32: '#f67c5f',
  64: '#f65e3b', 128: '#edcf72', 256: '#edcc61', 512: '#edc850',
  1024: '#edc53f', 2048: '#edc22e',
};
const POP_MS = 340; // 合并缩放/光晕持续时间
const SCORE_MS = 720; // 分数上浮消散时间

let state = Logic.initGame();
let roomRng = null; // 房间对局的 rng 实例（种子开局，每步复用同一实例，design-lock § 5）
let screen = 'guide'; // 'guide' | 'play' | 'room'：开局先显示玩法引导；room 为房间流程（含大厅/等待/对战/结算）

// 视觉特效状态
let mergeFx = []; // 活跃合并特效：{ cells:[{r,c,val}], t0 }
let particles = []; // 火花粒子：{ x,y,vx,vy,life,maxLife,color,size }
let scorePops = []; // 分数上浮：{ x,y,val,t0 }

// 震动开关：默认开；读取本地存储让用户偏好持久化
let vibrateOn = true;
```

```
if (isTT && tt.getStorageSync) {
  try {
    const v = tt.getStorageSync('vibrateOn');
    if (typeof v === 'boolean') vibrateOn = v;
  } catch (e) { /* 用默认值 */ }
}

// ----- 排行榜（自建云后端，全球榜） -----
const RANK_ENDPOINT = config.RANK_ENDPOINT;
const RANK_SECRET = config.RANK_SECRET || ''; // 防刷分签名密钥；为空则不上传签名（本地联调用）
let rankScroll = 0; // 榜单滚动偏移（像素）
let lastMoveY = 0; // 触摸滑动上一帧 Y
let rankLoading = false; // 榜单请求中
let rankError = null; // 榜单请求失败信息
let rankData = null; // 榜单数据：{ top, selfRank, selfName, selfScore }
let rankUid = ''; // 本机唯一标识（存 storage）
let rankSelfName = '我'; // 展示昵称（首次随机生成，存 storage）
let rankReturnScreen = 'play'; // 打开榜单前的界面，关闭时回到此处（健壮性）

if (isTT) {
  try {
    rankUid = tt.getStorageSync('rankUid') || '';
    rankSelfName = tt.getStorageSync('rankName') || '';
    if (!rankUid) {
      rankUid = 'u' + Date.now().toString(36) + Math.floor(Math.random() * 1e4).toString(36);
      tt.setStorageSync({ key: 'rankUid', data: rankUid });
    }
    if (!rankSelfName) {
      rankSelfName = '玩家' + Math.floor(1000 + Math.random() * 9000);
      tt.setStorageSync({ key: 'rankName', data: rankSelfName });
    }
  } catch (e) { /* 用默认值 */ }
}

// ----- 天梯（异步匹配 Phase 1）状态 -----
let ladderMatch = null; // 结算卡数据 { matchId, myScore, opponent, result, diff, synthetic }
let ladderLoading = false; // 匹配请求中
let ladderError = null; // 匹配请求失败信息
let ladderHist = null; // 战绩数据：{ list, total }
let ladderHistLoading = false; // 战绩请求中
let ladderHistError = null; // 战绩请求失败信息
let ladderScroll = 0; // 战绩滚动偏移（像素）
let lastMoveYL = 0; // 战绩列表滑动上一帧 Y
let moveSteps = 0; // 本局步数（用于天梯匹配上报）
let ladderSeq = 0; // 天梯请求序号：重开新局时自增，使进行中的旧回调失效（竞态守门）

// 对「uid|score|ts」做 HMAC 签名（仅当配置了 RANK_SECRET 时）
function signScore(uid, score, ts) {
  if (!RANK_SECRET) return '';
  return HMAC.hmacSha256Hex(RANK_SECRET, uid + '|' + score + '|' + ts);
}

// 上传当前分数到云后端（失败静默，不影响游戏）
function submitScore(s) {
  if (!isTT || !RANK_ENDPOINT) return;
  try {
    const ts = Math.floor(Date.now() / 1000);
    const sig = signScore(rankUid, s, ts);
    tt.request({
      url: RANK_ENDPOINT + '/score',
      method: 'POST',
      data: { uid: rankUid, name: rankSelfName, score: s, ts, sig },
    });
  }
}
```

```
    header: { 'Content-Type': 'application/json' },
    success: () => {},
    fail: () => {},
  });
} catch (e) { /* noop */ }
}

// 天梯：在游戏结束时上报本局成绩并发起异步匹配，成功后弹出结算卡（screen='ladder'）
function submitLadder(st) {
  if (!isTT || !RANK_ENDPOINT) return;
  try {
    const ts = Math.floor(Date.now() / 1000);
    const score = Math.floor(Number(st.score) || 0);
    const steps = Math.floor(Number(moveSteps) || 0);
    const seq = ++ladderSeq; // 本次天梯请求序号，用于竞态守门
    ladderLoading = true;
    ladderError = null;
    Ladder.fetchLadderMatch({
      uid: rankUid, name: rankSelfName, score, steps, boardSummary: st.grid, ts,
    }).then((resp) => {
      // 竞态守门：期间若已重开新局（ladderSeq 变化），丢弃本次回调结果，不覆盖新局。
      if (seq !== ladderSeq) { ladderLoading = false; return; }
      if (resp && resp.code === 0 && resp.data) {
        ladderMatch = resp.data;
        ladderError = null;
        screen = 'ladder';
      } else {
        ladderError = (resp && resp.message) || '天梯匹配失败';
      }
      ladderLoading = false;
    }).catch(() => {
      if (seq !== ladderSeq) { ladderLoading = false; return; }
      ladderError = '天梯请求失败，请检查网络';
      ladderLoading = false;
    });
  } catch (e) { /* noop */ }
}

// 打开天梯战绩面板：拉取本人历史（screen='ladderHistory'）
function openLadderHistory() {
  if (!isTT || !RANK_ENDPOINT) {
    ladderHistError = '未配置天梯地址：请在 config.js 填写 RANK_ENDPOINT';
    ladderHistLoading = false;
    return;
  }
  ladderHistLoading = true;
  ladderHistError = null;
  ladderHist = null;
  ladderScroll = 0;
  screen = 'ladderHistory';
  Ladder.fetchLadderHistory(rankUid, 30).then((resp) => {
    if (resp && resp.code === 0 && resp.data) {
      ladderHist = resp.data;
      ladderHistError = null;
    } else {
      ladderHistError = (resp && resp.message) || '战绩加载失败';
    }
    ladderHistLoading = false;
  }).catch(() => {
    ladderHistError = '战绩请求失败，请检查网络';
    ladderHistLoading = false;
  });
}
```

```

}

// 从云后端拉取榜单（前 100 + 自己名次）
function loadRank() {
  if (!isTT || !RANK_ENDPOINT) {
    rankError = '未配置榜单地址：请在 config.js 填写 RANK_ENDPOINT';
    rankLoading = false;
    return;
  }
  rankLoading = true;
  rankError = null;
  const ts = Math.floor(Date.now() / 1000);
  const sig = RANK_SECRET ? HMAC.hmacSha256Hex(RANK_SECRET, rankUid + '|' + ts) : '';
  tt.request({
    url: RANK_ENDPOINT + '/rank?uid=' + encodeURIComponent(rankUid) + '&limit=100&ts=' + ts + '&sig=' + encodeURIComponent(sig),
    method: 'GET',
    success: (res) => {
      if (res && res.data) { rankData = res.data; rankLoading = false; }
      else { rankError = '榜单数据格式异常'; rankLoading = false; }
    },
    fail: () => { rankError = '榜单请求失败，请检查网络或后端地址'; rankLoading = false; },
  });
}

// 打开排行榜：上传分数并拉榜
function openRank() {
  rankLoading = true;
  rankError = null;
  rankData = null;
  rankScroll = 0;
  rankReturnScreen = screen; // 记录来源界面，关闭时返回（避免跳错界面）
  screen = 'rank';
  submitScore(state.score);
  loadRank();
}

// 房间对战入口：进入大厅（hall）；由 Room 接管后续大厅/等待/对战/结算流程
function openRoom() {
  Room.open(rankUid, rankSelfName);
  screen = 'room';
}

// 房间入口按钮矩形（左上角，紧挨天梯按钮右侧，避开右上角系统胶囊，design-lock § 7）
function roomEntryBtnRect() {
  return { x: 166, y: 56, w: 74, h: 30 };
}

function rankBtnRect() {
  // 左上角（避开右上角系统胶囊），y 与排行榜面板顶边(p.y=56)平行对齐，不贴顶
  return { x: 12, y: 56, w: 74, h: 30 };
}

function rankPanelRect() {
  return { x: PAD, y: 56, w: W - PAD * 2, h: H - 56 - 24 };
}

function rankCloseRect() {
  // 放在面板左上角，避开抖音小游戏右上角的系统胶囊按钮（菜单/关闭），保证可点
  // 放大到 38px 并多留内边距，降低角落按钮误触/点不中的概率
  const p = rankPanelRect();
  return { x: p.x + 8, y: p.y + 8, w: 38, h: 38 };
}

function drawRank() {

```

```
const p = rankPanelRect();
ctx.fillStyle = 'rgba(0,0,0,0.72)';
ctx.fillRect(0, 0, W, H);
// 面板
ctx.save();
ctx.shadowColor = 'rgba(0,0,0,0.3)'; ctx.shadowBlur = 20; ctx.shadowOffsetY = 6;
ctx.fillStyle = 'faf8ef';
roundRect(p.x, p.y, p.w, p.h, 16); ctx.fill();
ctx.restore();
// 标题
ctx.fillStyle = '#776e65'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
ctx.font = 'bold 20px sans-serif';
ctx.fillText('全球排行榜 · 前 100 名', W / 2, p.y + 30);
// 关闭提示：系统返回键或左上角 × 均可关闭
ctx.fillStyle = 'bbada0'; ctx.font = '12px sans-serif';
ctx.fillText('系统返回键 / × 关闭', W / 2, p.y + 48);
// 关闭按钮 (×)
const cr = rankCloseRect();
ctx.strokeStyle = 'bbada0'; ctx.lineWidth = 2;
ctx.beginPath();
ctx.moveTo(cr.x + 8, cr.y + 8); ctx.lineTo(cr.x + cr.w - 8, cr.y + cr.h - 8);
ctx.moveTo(cr.x + cr.w - 8, cr.y + 8); ctx.lineTo(cr.x + 8, cr.y + cr.h - 8);
ctx.stroke();

// 加载中
if (rankLoading) {
  ctx.fillStyle = '#776e65'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
  ctx.font = '16px sans-serif';
  ctx.fillText('榜单加载中...', W / 2, p.y + p.h / 2);
  return;
}
// 出错
if (rankError) {
  ctx.fillStyle = '#c0392b'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
  ctx.font = '15px sans-serif';
  ctx.fillText(rankError, W / 2, p.y + p.h / 2 - 12);
  ctx.fillStyle = '#8f7a66'; ctx.font = 'bold 15px sans-serif';
  ctx.fillText('点击任意处重试', W / 2, p.y + p.h / 2 + 16);
  return;
}
// 无数据
if (!rankData) {
  ctx.fillStyle = '#776e65'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
  ctx.font = '16px sans-serif';
  ctx.fillText('暂无数据', W / 2, p.y + p.h / 2);
  return;
}

const view = rankData;
const listTop = p.y + 56;
const listBottom = p.y + p.h - 56;
const rowH = 34;
const contentH = view.top.length * rowH;
const maxScroll = Math.max(0, contentH - (listBottom - listTop));
rankScroll = Math.max(0, Math.min(maxScroll, rankScroll));

ctx.save();
ctx.beginPath(); ctx.rect(p.x, listTop, p.w, listBottom - listTop); ctx.clip();
for (let i = 0; i < view.top.length; i++) {
  const ry = listTop + i * rowH - rankScroll;
  if (ry + rowH < listTop || ry > listBottom) continue;
  const item = view.top[i];
```

```

    if (item.isSelf) {
      ctx.fillStyle = 'rgba(237,194,46,0.25)';
      roundRect(p.x + 8, ry + 2, p.w - 16, rowH - 4, 8); ctx.fill();
    }
    ctx.fillStyle = item.rank <= 3 ? '#edc22e' : '#776e65';
    ctx.textAlign = 'left'; ctx.textBaseline = 'middle';
    ctx.font = 'bold 15px sans-serif';
    ctx.fillText('#' + item.rank, p.x + 16, ry + rowH / 2);
    ctx.fillStyle = '#5b4a1f';
    ctx.font = '15px sans-serif';
    let nm = item.name; if (nm.length > 10) nm = nm.slice(0, 9) + '...';
    ctx.fillText(nm, p.x + 56, ry + rowH / 2);
    ctx.textAlign = 'right'; ctx.fillStyle = '#776e65'; ctx.font = 'bold 14px sans-serif';
    ctx.fillText(String(item.score), p.x + p.w - 16, ry + rowH / 2);
  }
  ctx.restore();

  // 底部固定条：始终显示「你的排名」
  const fy = p.y + p.h - 44;
  ctx.fillStyle = '#8f7a66';
  roundRect(p.x + 8, fy, p.w - 16, 34, 10); ctx.fill();
  ctx.fillStyle = 'white'; ctx.textAlign = 'left'; ctx.textBaseline = 'middle';
  ctx.font = 'bold 15px sans-serif';
  ctx.fillText('你的排名：第 ' + view.selfRank + ' 位', p.x + 20, fy + 17);
  ctx.textAlign = 'right'; ctx.font = '14px sans-serif';
  ctx.fillText('最高 ' + view.selfScore, p.x + p.w - 20, fy + 17);
}

// ----- 广告（占位，需真实 adUnitId） -----
const BANNER_AD_ID = config.BANNER_AD_ID; // ← 在 config.js 填写
const REWARD_AD_ID = config.REWARD_AD_ID; // ← 在 config.js 填写
let bannerAd = null;

function showBanner() {
  if (!isTT || !BANNER_AD_ID) return;
  try {
    if (!bannerAd) {
      bannerAd = tt.createBannerAd({
        adUnitId: BANNER_AD_ID,
        adIntervals: 30,
        style: { left: 0, top: H - 100, width: W },
      });
    }
    if (bannerAd.show) bannerAd.show().catch(() => {});
  } catch (e) {
    // 广告失败不应影响游戏
  }
}

function hideBanner() {
  if (bannerAd && bannerAd.hide) {
    try { bannerAd.hide(); } catch (e) { /* noop */ }
  }
}

function watchRewardedThenRestart() {
  const reset = () => {
    state = Logic.initGame();
    screen = 'play';
    moveSteps = 0;
    hideBanner();
    ladderSeq++; // 使任何进行中的旧天梯回调失效（竞态守门）
  }
}

```

```

    ladderLoading = false; // 一并清理天梯结算卡状态
    ladderMatch = null;
    ladderError = null;
  };
  if (!isTT || !REWARD_AD_ID) { reset(); return; }
  try {
    const ad = tt.createRewardedVideoAd({ adUnitId: REWARD_AD_ID });
    ad.onClose(reset);
    if (ad.show) ad.show().catch(reset);
  } catch (e) { reset(); }
}

function restart() {
  state = Logic.initGame();
  screen = 'play';
  moveSteps = 0;
  hideBanner();
  ladderSeq++; // 使任何进行中的旧天梯回调失效（竞态守门）
  ladderLoading = false; // 一并清理天梯结算卡状态
  ladderMatch = null;
  ladderError = null;
}

// ----- 房间对战：生命周期回调注入（Room 在 seed 开局 / 退出时回调 game.js） -----
Room.onBeginMatch((seed) => {
  const rng = Logic.makeRng(String(seed)); // 同一 seed → 双方同序列（design-lock § 5）
  state = Logic.initGame(rng);
  roomRng = rng; // 每步 move 必须复用同一 rng 实例（§ 5 硬约束）
  moveSteps = 0;
  ladderSeq++; // 使进行中的天梯回调失效（竞态守门）
  ladderMatch = null; ladderLoading = false; ladderError = null;
});
Room.onExit(() => {
  screen = 'play';
  state = Logic.initGame();
  moveSteps = 0;
  roomRng = null;
});

// 游戏结束：上报成绩 + 天梯结算 + 拉取个人排名（供结束遮罩展示「你的排名」）
// 排名拉取失败会静默降级（rankError/rankData 为空），绝不影响「再来一局」重开。
function triggerGameOver() {
  // 结束流程必须是「发射后不管」：任何一步失败（上报/匹配/拉榜）都不应阻断结束遮罩显示。
  try {
    submitScore(state.score); // fire-and-forget（内部 try/catch）
    showBanner(); // fire-and-forget（内部 try/catch）
    submitLadder(state); // fire-and-forget（内部 try/catch）
    rankData = null; // 清空旧榜单，触发加载态
    loadRank(); // 拉取个人排名（selfRank），失败静默降级
  } catch (e) {
    // 兜底：绝不让异常阻断「游戏结束」遮罩（state.over 已置位，draw 必画遮罩）
  }
}

// ----- 输入 -----
if (isTT) {
  let sx = 0;
  let sy = 0;
  tt.onTouchStart((e) => {
    sx = e.touches[0].clientX;
    sy = e.touches[0].clientY;
    lastMoveY = sy;
  });
}

```

```

    lastMoveYL = sy;
  });
  tt.onTouchEnd((e) => {
    const t = e.changedTouches[0];
    // 天梯结算卡：关闭 × / 天梯历史 / 再来一局 / 看战绩（均为轻点判定）
    if (screen === 'ladder') {
      const dx = t.clientX - sx, dy = t.clientY - sy;
      if (Math.abs(dx) < 12 && Math.abs(dy) < 12) {
        // 兜底态：screen==='ladder' 但天梯结算数据缺失且非加载/报错（空 match 黑屏场景）。
        // 此时结算卡无有效内容，点击任意处直接重开，绝不让玩家卡在空黑屏。
        if (!ladderMatch && !ladderLoading && !ladderError) {
          restart();
          return;
        }
        // 关闭 × 优先（错误态也不能吞掉关闭）
        const cr = Ladder.ladderCloseRect(L);
        if (hit(cr, t)) { screen = 'play'; return; }
        const hb = Ladder.ladderHistoryBtnRect(L);
        if (hit(hb, t)) { openLadderHistory(); return; }
        const ab = Ladder.ladderAgainBtnRect(L);
        if (hit(ab, t)) { restart(); return; }
        const rb = Ladder.ladderRecordsBtnRect(L);
        if (hit(rb, t)) { openLadderHistory(); return; }
        // 错误态：点任意处重试
        if (ladderError) { submitLadder(state); return; }
      }
      return;
    }
    // 天梯战绩面板：关闭 × 优先，错误态点击重试
    if (screen === 'ladderHistory') {
      const dx = t.clientX - sx, dy = t.clientY - sy;
      if (Math.abs(dx) < 12 && Math.abs(dy) < 12) {
        const cr = Ladder.ladderHistCloseRect(L);
        if (hit(cr, t)) { screen = 'play'; return; }
        if (ladderHistError) { openLadderHistory(); return; }
      }
      return;
    }
    // 排行榜界面：关闭按钮优先判定（任何状态、轻微滑动都可关），否则错误态点击重试
    if (screen === 'rank') {
      // 关闭判定不锁进「轻点阈值」：按下点(sx,sy)或抬起点(t)任一命中 × 即关，
      // 容忍手指从按下到抬起的轻微位移（触屏角落按钮常见），避免关闭被吞掉
      const cr = rankCloseRect();
      const pressPt = { clientX: sx, clientY: sy };
      if (hit(cr, t) || hit(cr, pressPt)) {
        screen = rankReturnScreen; // 回到打开榜单时的来源界面
        return;
      }
      // 错误态：轻点任意处重试（关闭已优先处理，这里不会再命中 ×）
      const dx = t.clientX - sx, dy = t.clientY - sy;
      if (Math.abs(dx) < 12 && Math.abs(dy) < 12) {
        if (rankError) { loadRank(); return; }
      }
      return;
    }
    if (screen === 'guide') {
      const bx = W / 2 - 80, by = H / 2 + 60, bw = 160, bh = 48;
      if (t.clientX >= bx && t.clientX <= bx + bw && t.clientY >= by && t.clientY <= by + bh) {
        screen = 'play';
      }
      return;
    }
  }

```



```

// 房间对战界面：轻点交给 Room 处理 UI 按钮；滑动则走棋盘移动（仅对战中）
if (screen === 'room') {
  const dx = t.clientX - sx, dy = t.clientY - sy;
  if (Math.abs(dx) < 16 && Math.abs(dy) < 16) {
    // 轻点：Room 处理大厅/等待/结算/键盘按钮（命中则已消费）
    if (Room.handleTouch(sx, sy, t)) return;
  }
  // 非轻点（滑动）且对战中 → 棋盘移动；本地不阻塞（design-lock § 6 ③）
  if (Room.getPhase() === 'playing' && !state.over) {
    const ddx = t.clientX - sx, ddy = t.clientY - sy;
    if (Math.abs(ddx) < 24 && Math.abs(ddy) < 24) return; // 视作点击，忽略
    let dir;
    if (Math.abs(ddx) > Math.abs(ddy)) dir = ddx > 0 ? 'right' : 'left';
    else dir = ddy > 0 ? 'down' : 'up';
    const res = Logic.move(state, dir, roomRng);
    if (res.moved) {
      state = res.state;
      moveSteps += 1;
      applyMoveFx(res);
      // 进度上报（节流 ≥500ms 或每步取先到者，design-lock § 4 Q4）
      Room.reportProgress(state.score, moveSteps, state.over);
      // 满格(over)或先到 2048 → 自动提交终局（won=false / won=true, § 3 Q1）
      const won = Logic.maxTile(state.grid) >= 2048;
      if (state.over || won) {
        Room.submitMyResult(state.score, moveSteps, won);
      }
    }
  }
  return;
}

// 震动开关按钮：命中即切换并返回，不触发移动
{
  const tg = vibrateToggleRect();
  if (t.clientX >= tg.x && t.clientX <= tg.x + tg.w &&
    t.clientY >= tg.y && t.clientY <= tg.y + tg.h) {
    vibrateOn = !vibrateOn;
    if (isTT && tt.setStorageSync) {
      try { tt.setStorageSync({ key: 'vibrateOn', data: vibrateOn }); } catch (e) { /* noop */ }
    }
  }
}

// ===== 文件边界：src/logic.js（接续上一部分） =====
// src/logic.js
// 合成能量（2048 式）核心逻辑——纯函数，无任何平台依赖，可在 Node 中直接单测。
// 抖音小游戏与 Node 均支持 CommonJS（require / module.exports）。

```

```
const SIZE = 4;
```

```
function createEmptyGrid() {
  const grid = [];
  for (let r = 0; r < SIZE; r++) grid.push(new Array(SIZE).fill(0));
  return grid;
}
```

```
function cloneGrid(grid) {
  return grid.map((row) => row.slice());
}
```

```
function getEmptyCells(grid) {
  const cells = [];
  for (let r = 0; r < SIZE; r++)
    for (let c = 0; c < SIZE; c++)
      if (grid[r][c] === 0) cells.push([r, c]);
}
```

```
    return cells;
  }

// 在随机空格生成新棋子：rng() 返回 [0,1)，rng() < 0.9 出 2，否则出 4
function spawnTile(grid, rng) {
  rng = rng || Math.random;
  const cells = getEmptyCells(grid);
  if (cells.length === 0) return false;
  const [r, c] = cells[Math.floor(rng() * cells.length)];
  grid[r][c] = rng() < 0.9 ? 2 : 4;
  return true;
}

function initGame(rng) {
  const grid = createEmptyGrid();
  spawnTile(grid, rng);
  spawnTile(grid, rng);
  return { grid, score: 0, over: false };
}

// 将一行向左压缩并合并：返回 { row, gained }
function compress(row) {
  const filtered = row.filter((v) => v !== 0);
  const result = [];
  const mergedAt = [];
  let gained = 0;
  for (let i = 0; i < filtered.length; i++) {
    if (i + 1 < filtered.length && filtered[i] === filtered[i + 1]) {
      const merged = filtered[i] * 2;
      result.push(merged);
      mergedAt.push(result.length - 1);
      gained += merged;
      i++; // 跳过被合并的下一个
    } else {
      result.push(filtered[i]);
    }
  }
  while (result.length < SIZE) result.push(0);
  return { row: result, gained, mergedAt };
}

// 顺时针旋转 grid times 次 (90 ° /次)，使任意方向统一成「向左」处理
function rotate(grid, times) {
  let g = cloneGrid(grid);
  const t = ((times % 4) + 4) % 4;
  for (let k = 0; k < t; k++) {
    const ng = createEmptyGrid();
    for (let r = 0; r < SIZE; r++)
      for (let c = 0; c < SIZE; c++) ng[c][SIZE - 1 - r] = g[r][c];
    g = ng;
  }
  return g;
}

// 将坐标 (r,c) 顺时针旋转 t 次，映射回原坐标系（用于把「向左压缩」后的合并位置还原到真实棋盘坐标）
function mapCoord(r, c, t) {
  let nr = r, nc = c;
  const k = ((t % 4) + 4) % 4;
  for (let i = 0; i < k; i++) {
    const tr = nc;
    const tc = SIZE - 1 - nr;
    nr = tr; nc = tc;
  }
}
```

```
}
return [nr, nc];
}

// direction: 'left' | 'right' | 'up' | 'down'
const ROT_MAP = { left: 0, up: 3, right: 2, down: 1 };

function move(state, direction, rng) {
  rng = rng || Math.random;
  if (state.over) return { moved: false };
  const times = ROT_MAP[direction] || 0;
  const rotated = rotate(state.grid, times);

  let gainedTotal = 0;
  let moved = false;
  const newRows = [];
  const mergedRot = [];
  for (let r = 0; r < SIZE; r++) {
    const { row, gained, mergedAt } = compress(rotated[r]);
    gainedTotal += gained;
    for (const col of mergedAt) mergedRot.push([r, col]);
    if (row.some((v, i) => v !== rotated[r][i])) moved = true;
    newRows.push(row);
  }

  // 棋盘未变：仍需判定是否已无路可走（满格且无相邻可合并）
  if (!moved) {
    return { moved: false, over: !canMove(state.grid) };
  }

  const t = (4 - times) % 4;
  const newGrid = rotate(newRows, t);
  const merged = mergedRot.map(([r, c]) => mapCoord(r, c, t));
  const score = state.score + gainedTotal;

  // 移动后已无路可走（满格且无相邻可合并）：直接判负，不生成新棋子
  if (!canMove(newGrid)) {
    const newState = { grid: newGrid, score, over: true };
    return { moved: true, state: newState, gained: gainedTotal, merged };
  }

  // 仍可继续：生成新棋子，并重新判定生成后是否恰好满格死局
  const newState = { grid: newGrid, score };
  spawnTile(newState.grid, rng);
  newState.over = !canMove(newState.grid);
  return { moved: true, state: newState, gained: gainedTotal, merged };
}

// 是否还能移动：有空格 或 存在相邻相等
function canMove(grid) {
  if (getEmptyCells(grid).length > 0) return true;
  for (let r = 0; r < SIZE; r++) {
    for (let c = 0; c < SIZE; c++) {
      const v = grid[r][c];
      if (c + 1 < SIZE && grid[r][c + 1] === v) return true;
      if (r + 1 < SIZE && grid[r + 1][c] === v) return true;
    }
  }
  return false;
}

function maxTile(grid) {
```

```
let m = 0;
for (const row of grid) for (const v of row) if (v > m) m = v;
return m;
}

// ----- Phase 2：种子随机数（design-lock § 5.1，零依赖，纯 JS） -----
// 同 seed → 同一 PRNG 序列：双方用同一服务端 seed 开局，棋盘起点一致、RNG 零优势。
// 客户端每次 Logic.move 必须传入「同一个 rng 实例」（见 src/room.js）。

// xmur3：将任意字符串哈希为 32 位种子初值（确定性）
function xmur3(str) {
  let h = 1779033703 ^ str.length;
  for (let i = 0; i < str.length; i++) {
    h = Math.imul(h ^ str.charCodeAt(i), 3432918353);
    h = (h << 13) | (h >>> 19);
  }
  return function () {
    h = Math.imul(h ^ (h >>> 16), 2246822507);
    h = Math.imul(h ^ (h >>> 13), 3266489909);
    h ^= h >>> 16;
    return h >>> 0;
  };
}

// mulberry32：32 位种子 → [0,1) 均匀分布伪随机序列（确定性）
function mulberry32(a) {
  return function () {
    a |= 0; a = (a + 0x6D2B79F5) | 0;
    let t = Math.imul(a ^ (a >>> 15), 1 | a);
    t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t;
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}

// makeRng：将种子字符串转为可注入 Logic 的 rng 函数
function makeRng(seedStr) {
  const seedFn = xmur3(String(seedStr));
  return mulberry32(seedFn());
}

module.exports = {
  SIZE,
  createEmptyGrid,
  cloneGrid,
  getEmptyCells,
  spawnTile,
  initGame,
  compress,
  rotate,
  move,
  canMove,
  maxTile,
  makeRng,
};

// ===== 文件边界：src/hmac.js（接续上一部分） =====
// src/hmac.js —— 纯 JS 实现 SHA-256 + HMAC-SHA256（抖音小游戏运行时无 Node crypto，需自带）
// 同时支持 CommonJS(require) 与浏览器/小游戏全局。
// 用途：客户端对「上报分数」做 HMAC 签名，后端用相同密钥验签，防止随意伪造高分。
(function (root, factory) {
  if (typeof module === 'object' && module.exports) module.exports = factory();
  else root.HMAC = factory();
})(typeof self !== 'undefined' ? self : this, function () {
```

```
function rotr(x, n) { return (x >>> n) | (x << (32 - n)); }
```

```
const K = new Uint32Array([
  0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
  0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
  0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
  0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
  0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13, 0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
  0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
  0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5, 0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
  0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90befffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2,
]);
```

```
// 返回原始 32 字节摘要
```

```
function sha256Raw(bytes) {
  const Hh = [0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a, 0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19];
  const l = bytes.length;
  const bitLen = l * 8;
  const withOne = l + 1;
  const pad = (56 - (withOne % 64) + 64) % 64;
  const total = withOne + pad + 8;
  const msg = new Uint8Array(total);
  msg.set(bytes, 0);
  msg[l] = 0x80;
  const dv = new DataView(msg.buffer);
  dv.setUint32(total - 4, bitLen >>> 0, false);
  dv.setUint32(total - 8, Math.floor(bitLen / 0x100000000) >>> 0, false);
```

```
const w = new Uint32Array(64);
for (let off = 0; off < total; off += 64) {
  for (let i = 0; i < 16; i++) w[i] = dv.getUint32(off + i * 4, false);
  for (let i = 16; i < 64; i++) {
    const s0 = rotr(w[i - 15], 7) ^ rotr(w[i - 15], 18) ^ (w[i - 15] >>> 3);
    const s1 = rotr(w[i - 2], 17) ^ rotr(w[i - 2], 19) ^ (w[i - 2] >>> 10);
    w[i] = (w[i - 16] + s0 + w[i - 7] + s1) >>> 0;
  }
  let a = Hh[0], b = Hh[1], c = Hh[2], d = Hh[3], e = Hh[4], f = Hh[5], g = Hh[6], h = Hh[7];
  for (let i = 0; i < 64; i++) {
    const S1 = rotr(e, 6) ^ rotr(e, 11) ^ rotr(e, 25);
    const ch = (e & f) ^ (~e & g);
    const t1 = (h + S1 + ch + K[i] + w[i]) >>> 0;
    const S0 = rotr(a, 2) ^ rotr(a, 13) ^ rotr(a, 22);
    const maj = (a & b) ^ (a & c) ^ (b & c);
    const t2 = (S0 + maj) >>> 0;
    h = g; g = f; f = e; e = (d + t1) >>> 0; d = c; c = b; b = a; a = (t1 + t2) >>> 0;
  }
  Hh[0] = (Hh[0] + a) >>> 0; Hh[1] = (Hh[1] + b) >>> 0; Hh[2] = (Hh[2] + c) >>> 0;
  Hh[3] = (Hh[3] + d) >>> 0; Hh[4] = (Hh[4] + e) >>> 0; Hh[5] = (Hh[5] + f) >>> 0;
  Hh[6] = (Hh[6] + g) >>> 0; Hh[7] = (Hh[7] + h) >>> 0;
}
const out = new Uint8Array(32);
for (let i = 0; i < 8; i++) {
  out[i * 4] = (Hh[i] >>> 24) & 0xff;
  out[i * 4 + 1] = (Hh[i] >>> 16) & 0xff;
  out[i * 4 + 2] = (Hh[i] >>> 8) & 0xff;
  out[i * 4 + 3] = Hh[i] & 0xff;
}
return out;
}
```

```
function strToBytes(s) {
  const b = new Uint8Array(s.length);
```

```
for (let i = 0; i < s.length; i++) b[i] = s.charCodeAt(i) & 0xff;
return b;
}

function toHex(bytes) {
  let s = '';
  for (let i = 0; i < bytes.length; i++) s += ('0' + bytes[i].toString(16)).slice(-2);
  return s;
}

function hmacSha256Hex(key, msg) {
  const blockSize = 64;
  let keyBytes = strToBytes(key);
  if (keyBytes.length > blockSize) keyBytes = sha256Raw(keyBytes); // 密钥过长：先哈希成 32 字节
  const oKey = new Uint8Array(blockSize);
  const iKey = new Uint8Array(blockSize);
  for (let i = 0; i < blockSize; i++) {
    const kb = i < keyBytes.length ? keyBytes[i] : 0;
    oKey[i] = kb ^ 0x5c;
    iKey[i] = kb ^ 0x36;
  }
  const inner = new Uint8Array(blockSize + msg.length);
  inner.set(iKey, 0);
  inner.set(strToBytes(msg), blockSize);
  const innerHash = sha256Raw(inner); // 32 字节原始摘要
  const outer = new Uint8Array(blockSize + 32);
  outer.set(oKey, 0);
  outer.set(innerHash, blockSize);
  return toHex(sha256Raw(outer));
}

return {
  sha256Hex: (s) => toHex(sha256Raw(strToBytes(s))),
  hmacSha256Hex,
};
});
// ===== 文件边界：src/ladder.js（接续上一部分） =====
// src/ladder.js —— 天梯（异步匹配）前端客户端 + Canvas UI（零依赖，原生 tt + Canvas）
// 抖音小游戏运行时无 Node crypto，签名复用 src/hmac.js 的纯 JS HMAC-SHA256。
//
// 约定（与后端严格一致）：
// - 字段 snake_case；ts 秒级 Unix；HMAC-SHA256 密钥 RANK_SECRET；canonical 用 | 拼接。
// - match 签名 canonical = uid|score|steps|ts
// - history 签名 canonical = uid|ts
// - 所有自定义按钮避开右上角系统胶囊：关闭×与“天梯历史”入口放左上角。
//
// 该模块在 Node 下也可被安全 require（仅定义函数/常量，不触碰 tt 全局），便于 QA。

const HMAC = require('./hmac.js');
const cfg = require('./config.js');

const isTT = typeof tt !== 'undefined' && !!tt.createCanvas;

// ----- 签名 -----
function signLadder(uid, score, steps, ts) {
  if (!cfg.RANK_SECRET) return '';
  return HMAC.hmacSha256Hex(
    cfg.RANK_SECRET,
    String(uid) + '|' + Math.floor(Number(score) || 0) + '|' + Math.floor(Number(steps) || 0) + '|' + ts
  );
}
function signLadderHistory(uid, ts) {
```

```

    if (!cfg.RANK_SECRET) return '';
    return HMAC.hmacSha256Hex(cfg.RANK_SECRET, String(uid) + '|' + ts);
}

// ----- 网络请求 -----
function fetchLadderMatch(payload) {
    return new Promise((resolve, reject) => {
        if (!isTT || !cfg.RANK_ENDPOINT) { reject(new Error('未配置天梯后端')); return; }
        const sig = signLadder(payload.uid, payload.score, payload.steps, payload.ts);
        tt.request({
            url: cfg.LADDER_MATCH_PATH,
            method: 'POST',
            data: Object.assign({}, payload, { sig }),
            header: { 'Content-Type': 'application/json' },
            success: (res) => resolve(res && res.data ? res.data : { code: 5, data: null, message: 'empty response' }),
            fail: (e) => reject(e),
        });
    });
}

function fetchLadderHistory(uid, limit) {
    return new Promise((resolve, reject) => {
        if (!isTT || !cfg.RANK_ENDPOINT) { reject(new Error('未配置天梯后端')); return; }
        const ts = Math.floor(Date.now() / 1000);
        const sig = signLadderHistory(uid, ts);
        const q = 'uid=' + encodeURIComponent(uid) +
            '&limit=' + (limit || 20) +
            '&ts=' + ts +
            '&sig=' + encodeURIComponent(sig);
        tt.request({
            url: cfg.LADDER_HISTORY_PATH + '?' + q,
            method: 'GET',
            success: (res) => resolve(res && res.data ? res.data : { code: 5, data: null, message: 'empty response' }),
            fail: (e) => reject(e),
        });
    });
}

// ===== Canvas UI =====
// 绘制函数均接收 ctx 与布局对象 L = { W, H, PAD, boardX, boardY, boardSize, cell },
// 不依赖 game.js 内部变量，保持本模块自治。矩形位置由各 rect 导出供点击判定复用（单一数据源）。

const TILE_COLORS = [
    2: '#eee4da', 4: '#ede0c8', 8: '#f2b179', 16: '#f59563', 32: '#f67c5f',
    64: '#f65e3b', 128: '#edcf72', 256: '#edcc61', 512: '#edc850',
    1024: '#edc53f', 2048: '#edc22e',
];

function rr(ctx, x, y, w, h, r) {
    ctx.beginPath();
    ctx.moveTo(x + r, y);
    ctx.arcTo(x + w, y, x + w, y + h, r);
    ctx.arcTo(x + w, y + h, x, y + h, r);
    ctx.arcTo(x, y + h, x, y, r);
    ctx.arcTo(x, y, x + w, y, r);
    ctx.closePath();
}

function ladderEntryBtnRect() {
    // 游戏进行中：左上角，紧挨排行榜按钮右侧（避开右上角系统胶囊）
    return { x: 92, y: 56, w: 74, h: 30 };
}

```

```

function ladderPanelRect(L) {
  return { x: L.PAD, y: 56, w: L.W - L.PAD * 2, h: L.H - 56 - 24 };
}
function ladderCloseRect(L) {
  // 面板左上角 ×（避开系统胶囊）
  const p = ladderPanelRect(L);
  return { x: p.x + 10, y: p.y + 10, w: 28, h: 28 };
}
function ladderHistoryBtnRect(L) {
  // 面板左上角 “天梯历史” 入口（紧随 × 右侧）
  const p = ladderPanelRect(L);
  return { x: p.x + 10 + 28 + 8, y: p.y + 10, w: 92, h: 28 };
}
function ladderAgainBtnRect(L) {
  // 结算卡底部 “再来一局”
  const p = ladderPanelRect(L);
  return { x: p.x + 16, y: p.y + p.h - 52, w: (p.w - 16 * 3) / 2, h: 40 };
}
function ladderRecordsBtnRect(L) {
  // 结算卡底部 “看战绩”
  const p = ladderPanelRect(L);
  return { x: p.x + 16 * 2 + (p.w - 16 * 3) / 2, y: p.y + p.h - 52, w: (p.w - 16 * 3) / 2, h: 40 };
}
function ladderHistPanelRect(L) { return ladderPanelRect(L); }
function ladderHistCloseRect(L) {
  const p = ladderHistPanelRect(L);
  return { x: p.x + 10, y: p.y + 10, w: 28, h: 28 };
}

function drawMiniBoard(ctx, x, y, w, grid) {
  const n = grid.length;
  if (!n) return;
  const pad = 4;
  const cw = (w - pad * (n + 1)) / n;
  for (let r = 0; r < n; r++) {
    for (let c = 0; c < n; c++) {
      const v = grid[r][c];
      const px = x + pad + c * (cw + pad);
      const py = y + pad + r * (cw + pad);
      ctx.fillStyle = v ? (TILE_COLORS[v] || '#3c3a32') : '#cdc1b4';
      rr(ctx, px, py, cw, cw, cw * 0.12); ctx.fill();
      if (v) {
        ctx.fillStyle = v <= 4 ? '#776e65' : '#f9f6f2';
        ctx.font = 'bold ' + Math.floor(cw * 0.42) + 'px sans-serif';
        ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
        ctx.fillText(String(v), px + cw / 2, py + cw / 2);
      }
    }
  }
}

function drawPlayer(ctx, x, y, w, name, score, boardSummary, synthetic) {
  // 头像占位（圆）
  const cx = x + 24, cy = y + 24, r = 22;
  ctx.fillStyle = synthetic ? '#b9aaa0' : '#8f7a66';
  ctx.beginPath(); ctx.arc(cx, cy, r, 0, Math.PI * 2); ctx.fill();
  ctx.fillStyle = '#fff'; ctx.font = 'bold 20px sans-serif';
  ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
  ctx.fillText(synthetic ? '?' : (name ? String(name).slice(0, 1) : '?'), cx, cy);
  // 昵称
  ctx.fillStyle = '#5b4a1f'; ctx.textAlign = 'left'; ctx.textBaseline = 'middle';
  ctx.font = 'bold 15px sans-serif';

```



```

let nm = String(name || '玩家'); if (nm.length > 8) nm = nm.slice(0, 7) + '...';
ctx.fillText(nm, x + 54, y + 16);
// 分数
ctx.fillStyle = '#776e65'; ctx.font = 'bold 20px sans-serif';
ctx.fillText(String(score), x + 54, y + 40);
// 迷你棋盘（若有 boardSummary）
if (Array.isArray(boardSummary) && boardSummary.length) {
  drawMiniBoard(ctx, x, y + 66, w, boardSummary);
}
}

// 绘制天梯结算卡（你 VS 对手）。返回 void。
function drawLadderCard(ctx, L, match, opts) {
  opts = opts || {};
  const p = ladderPanelRect(L);
  ctx.fillStyle = 'rgba(0,0,0,0.72)';
  ctx.fillRect(0, 0, L.W, L.H);
  ctx.save();
  ctx.shadowColor = 'rgba(0,0,0,0.3)'; ctx.shadowBlur = 20; ctx.shadowOffsetY = 6;
  ctx.fillStyle = '#faf8e7';
  rr(ctx, p.x, p.y, p.w, p.h, 16); ctx.fill();
  ctx.restore();

  // 关闭 ×（左上角，避开系统胶囊）
  const cr = ladderCloseRect(L);
  ctx.strokeStyle = 'bbada0'; ctx.lineWidth = 2;
  ctx.beginPath();
  ctx.moveTo(cr.x + 8, cr.y + 8); ctx.lineTo(cr.x + cr.w - 8, cr.y + cr.h - 8);
  ctx.moveTo(cr.x + cr.w - 8, cr.y + 8); ctx.lineTo(cr.x + 8, cr.y + cr.h - 8);
  ctx.stroke();
  // “天梯历史”入口（左上角，紧随 ×）
  const hb = ladderHistoryBtnRect(L);
  ctx.fillStyle = '#8f7a66';
  rr(ctx, hb.x, hb.y, hb.w, hb.h, hb.h / 2); ctx.fill();
  ctx.fillStyle = '#fff'; ctx.font = 'bold 13px sans-serif';
  ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
  ctx.fillText('天梯历史', hb.x + hb.w / 2, hb.y + hb.h / 2);

  // 加载中
  if (opts.loading) {
    ctx.fillStyle = '#776e65'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
    ctx.font = '16px sans-serif';
    ctx.fillText('天梯匹配中...', L.W / 2, p.y + p.h / 2);
    return;
  }
  // 出错
  if (opts.error) {
    ctx.fillStyle = '#c0392b'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
    ctx.font = '15px sans-serif';
    ctx.fillText(opts.error, L.W / 2, p.y + p.h / 2 - 12);
    ctx.fillStyle = '#8f7a66'; ctx.font = 'bold 15px sans-serif';
    ctx.fillText('点击任意处重试', L.W / 2, p.y + p.h / 2 + 16);
    return;
  }
  // 兜底：天梯结算数据缺失或结构异常（网络/后端返回不符、空数据、缺 opponent 等）。
  // 绝不允许裸 return 留黑屏——必须给出明确的结束提示与重开入口，
  // 保证玩家满格后一定能看到提示并重新开局。
  const canRenderMatch = match && match.opponent && typeof match.myScore === 'number';
  if (!canRenderMatch) {
    drawLadderFallback(ctx, L, opts);
    return;
  }
}

```

```
// 标题
ctx.fillStyle = '#776e65'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
ctx.font = 'bold 20px sans-serif';
ctx.fillText('你 VS 对手', L.W / 2, p.y + 30);
ctx.fillStyle = '#bbada0'; ctx.font = '12px sans-serif';
ctx.fillText('系统返回键 / × 关闭', L.W / 2, p.y + 48);

// 双方信息
const cardY = p.y + 72;
const colW = (p.w - 40) / 2;
drawPlayer(ctx, p.x + 16, cardY, colW, '你', match.myScore, null, false);
// VS
ctx.fillStyle = '#edc22e'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
ctx.font = 'bold 22px sans-serif';
ctx.fillText('VS', L.W / 2, cardY + 40);
drawPlayer(ctx, p.x + p.w - 16 - colW, cardY, colW,
  match.opponent.name, match.opponent.score, match.opponent.boardSummary, match.opponent.synthetic);

// 结果
const resY = cardY + 100;
const resText = match.result === 'win'? '胜利!' : match.result === 'loss'? '惜败!' : '平局';
const resColor = match.result === 'win'? '#3a8a3a': match.result === 'loss'? '#c0392b': '#8f7a66';
ctx.fillStyle = resColor; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
ctx.font = 'bold 26px sans-serif';
ctx.fillText(resText, L.W / 2, resY);
ctx.fillStyle = '#776e65'; ctx.font = '16px sans-serif';
const sign = match.diff > 0? '+': '';
ctx.fillText('分差' + sign + match.diff + (match.synthetic? '（合成对手）': ''), L.W / 2, resY + 28);

// 个人排名（游戏结束时已拉取，仅在有效正整数时展示）
if (opts.selfRank && Number(opts.selfRank) > 0) {
  ctx.fillStyle = '#edc22e'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
  ctx.font = '15px sans-serif';
  ctx.fillText('你的排名：第' + opts.selfRank + '位', L.W / 2, resY + 54);
}

// 底部按钮：再来一局 / 看战绩
const ab = ladderAgainBtnRect(L);
ctx.fillStyle = '#edc22e'; rr(ctx, ab.x, ab.y, ab.w, ab.h, ab.h / 2); ctx.fill();
ctx.fillStyle = '#5b4a1f'; ctx.font = 'bold 15px sans-serif';
ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
ctx.fillText('再来一局', ab.x + ab.w / 2, ab.y + ab.h / 2);
const rb = ladderRecordsBtnRect(L);
ctx.fillStyle = '#8f7a66'; rr(ctx, rb.x, rb.y, rb.w, rb.h, rb.h / 2); ctx.fill();
ctx.fillStyle = '#fff'; ctx.font = 'bold 15px sans-serif';
ctx.fillText('看战绩', rb.x + rb.w / 2, rb.y + rb.h / 2);
}

// 天梯结算卡的兜底渲染：当 match 缺失或结构异常（空数据/网络超时/后端返回不符）时调用。
// 此时面板与 × 已由 drawLadderCard 先行绘制，这里只补充「结束提示 + 重开入口」，
// 确保玩家满格后绝不卡在空黑屏，且可通过点击任意处重开（见 game.js 触摸兜底分支）。
function drawLadderFallback(ctx, L, opts) {
  opts = opts || {};
  const p = ladderPanelRect(L);
  const cy = p.y + p.h / 2;
  // 标题：游戏结束
  ctx.fillStyle = '#776e65'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
  ctx.font = 'bold 22px sans-serif';
  ctx.fillText('游戏结束', L.W / 2, cy - 56);
  // 本局得分（由 game.js 透传，缺省 0）
  const score = (opts.score != null)? opts.score : 0;
```

```

ctx.fillStyle = '#5b4a1f'; ctx.font = 'bold 18px sans-serif';
ctx.fillText('本局得分：' + score, L.W / 2, cy - 24);
// 说明
ctx.fillStyle = '#776e65'; ctx.font = '15px sans-serif';
ctx.fillText('天梯结算暂不可用', L.W / 2, cy + 4);
// 个人排名（若已拉取到有效名次）
if (opts.selfRank && Number(opts.selfRank) > 0) {
  ctx.fillStyle = '#edc22e'; ctx.font = '15px sans-serif';
  ctx.fillText('你的排名：第' + opts.selfRank + '位', L.W / 2, cy + 30);
}
// 重开提示（点击任意处 → restart，见 game.js）
ctx.fillStyle = '#8f7a66'; ctx.font = 'bold 16px sans-serif';
ctx.fillText('点击任意处再来一局', L.W / 2, cy + 56);
}

function formatTs(ts) {
  const t = Number(ts) || 0;
  if (!t) return '';
  const d = new Date(t * 1000);
  const pad = (n) => (n < 10 ? '0' : '') + n;
  return (d.getMonth() + 1) + '-' + pad(d.getDate()) + ' ' + pad(d.getHours()) + ':' + pad(d.getMinutes());
}

// 绘制天梯战绩面板。返回裁剪后的 scroll（供 game.js 写回状态）。
function drawLadderHistory(ctx, L, hist, opts) {
  opts = opts || {};
  const p = ladderHistPanelRect(L);
  ctx.fillStyle = 'rgba(0,0,0,0.72)';
  ctx.fillRect(0, 0, L.W, L.H);
  ctx.save();
  ctx.shadowColor = 'rgba(0,0,0,0.3)'; ctx.shadowBlur = 20; ctx.shadowOffsetY = 6;
  ctx.fillStyle = '#faf8ef';
  rr(ctx, p.x, p.y, p.w, p.h, 16); ctx.fill();
  ctx.restore();

  // × 关闭（左上角）
  const cr = ladderHistCloseRect(L);
  ctx.strokeStyle = '#bbada0'; ctx.lineWidth = 2;
  ctx.beginPath();
  ctx.moveTo(cr.x + 8, cr.y + 8); ctx.lineTo(cr.x + cr.w - 8, cr.y + cr.h - 8);
  ctx.moveTo(cr.x + cr.w - 8, cr.y + 8); ctx.lineTo(cr.x + 8, cr.y + cr.h - 8);
  ctx.stroke();

  // 标题
  ctx.fillStyle = '#776e65'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
  ctx.font = 'bold 20px sans-serif';
  ctx.fillText('天梯战绩', L.W / 2, p.y + 30);
  ctx.fillStyle = '#bbada0'; ctx.font = '12px sans-serif';
  ctx.fillText('系统返回键 / × 关闭', L.W / 2, p.y + 48);

  if (opts.loading) {
    ctx.fillStyle = '#776e65'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
    ctx.font = '16px sans-serif';
    ctx.fillText('战绩加载中...', L.W / 2, p.y + p.h / 2);
    return opts.scroll || 0;
  }
  if (opts.error) {
    ctx.fillStyle = '#c0392b'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
    ctx.font = '15px sans-serif';
    ctx.fillText(opts.error, L.W / 2, p.y + p.h / 2 - 12);
    ctx.fillStyle = '#8f7a66'; ctx.font = 'bold 15px sans-serif';
    ctx.fillText('点击任意处重试', L.W / 2, p.y + p.h / 2 + 16);
  }
}

```

```

    return opts.scroll || 0;
  }
  if (!hist || !hist.list || !hist.list.length) {
    ctx.fillStyle = '#776e65'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
    ctx.font = '16px sans-serif';
    ctx.fillText('暂无战绩', L.W / 2, p.y + p.h / 2);
    return opts.scroll || 0;
  }

  const list = hist.list;
  const listTop = p.y + 64;
  const listBottom = p.y + p.h - 24;
  const rowH = 46;
  const contentH = list.length * rowH;
  const maxScroll = Math.max(0, contentH - (listBottom - listTop));
  const scroll = Math.max(0, Math.min(maxScroll, opts.scroll || 0));

  ctx.save();
  ctx.beginPath(); ctx.rect(p.x, listTop, p.w, listBottom - listTop); ctx.clip();
  for (let i = 0; i < list.length; i++) {
    const ry = listTop + i * rowH - scroll;
    if (ry + rowH < listTop || ry > listBottom) continue;
    const it = list[i];
    const rx = p.x + 10;
    const rw = p.w - 20;
    const rc = it.result === 'win' ? '#3a8a3a' : it.result === 'loss' ? '#c0392b' : '#8f7a66';
    ctx.fillStyle = 'rgba(0,0,0,0.04)';
    rr(ctx, rx, ry + 4, rw, rowH - 8, 8); ctx.fill();
    // 结果色条
    ctx.fillStyle = rc;
    ctx.fillRect(rx, ry + 4, 4, rowH - 8);
    // 对手名 + 时间
    ctx.fillStyle = '#5b4a1f'; ctx.textAlign = 'left'; ctx.textBaseline = 'middle';
    ctx.font = 'bold 14px sans-serif';
    let nm = String(it.oppname || '对手'); if (nm.length > 8) nm = nm.slice(0, 7) + '...';
    ctx.fillText(nm + (it.opponentSynthetic ? '（合成）' : ''), rx + 14, ry + rowH / 2 - 9);
    ctx.fillStyle = '#a89b8c'; ctx.font = '11px sans-serif';
    ctx.fillText(formatTs(it.ts), rx + 14, ry + rowH / 2 + 9);
    // 比分 + 结果
    ctx.fillStyle = '#776e65'; ctx.textAlign = 'right'; ctx.font = 'bold 13px sans-serif';
    const sign = (it.diff > 0 ? '+' : '');
    ctx.fillText(it.myScore + ' : ' + it.oppScore, rx + rw - 14, ry + rowH / 2 - 9);
    ctx.fillStyle = rc; ctx.font = 'bold 13px sans-serif';
    const rt = it.result === 'win' ? '胜' : it.result === 'loss' ? '负' : '平';
    ctx.fillText(rt + ' ' + sign + it.diff, rx + rw - 14, ry + rowH / 2 + 9);
  }
  ctx.restore();
  return scroll;
}

module.exports = {
  // 客户端
  signLadder,
  signLadderHistory,
  fetchLadderMatch,
  fetchLadderHistory,
  // 绘制
  drawLadderCard,
  drawLadderHistory,
  drawMiniBoard,
  // 矩形（点击判定复用）
  ladderEntryBtnRect,

```

```

ladderPanelRect,
ladderCloseRect,
ladderHistoryBtnRect,
ladderAgainBtnRect,
ladderRecordsBtnRect,
ladderHistPanelRect,
ladderHistCloseRect,
};
// ===== 文件边界：src/room.js（接续上一部分） =====
// src/room.js —— 房间对战前端客户端 + Canvas UI（零依赖，原生 tt + Canvas）
// 抖音小游戏运行时无 Node crypto，签名复用 src/hmac.js 的纯 JS HMAC-SHA256。
//
// 约定（与后端 / design-lock 严格一致）：
// - 字段 snake_case；ts 秒级 Unix；HMAC-SHA256 密钥 RANK_SECRET；canonical 用 | 拼接。
// - create/join/state/leave/reset 签名 canonical = uid|ts
// - progress/result 签名 canonical = uid|score|steps|ts
// - 色板复用 #edc22e(金) / #5b4a1f(深棕)（design-lock § 7）。
// - 所有自定义按钮避开右上角系统胶囊：大厅/等待/结算卡的「退出/返回」放左上角。
//
// 该模块在 Node 下也可被安全 require（仅定义函数/常量，不触碰 tt 全局），便于 QA。
// 轮询容错三原则（design-lock § 6）已落实：
// ① 超时 race（withTimeout 3500ms）② 指数退避（1500→3000→4500→6000，成功复位）③ 本地不阻塞。

const HMAC = require('./hmac.js');
const cfg = require('../config.js');
const Logic = require('./logic.js');

const isTT = typeof tt !== 'undefined' && !!tt.createCanvas;

// ----- 设计锁定常量（design-lock § 1.2） -----
const ROOM_POLL_MS = cfg.ROOM_POLL_MS || 1500;
const ROOM_WAIT_POLL_MS = cfg.ROOM_WAIT_POLL_MS || 1000;
const ROOM_TIMEOUT_RACE_MS = cfg.ROOM_TIMEOUT_RACE_MS || 3500;
const ROOM_BACKOFF_MS = cfg.ROOM_BACKOFF_MS || [1500, 3000, 4500, 6000];
const ROOM_BACKOFF_MAX_MS = cfg.ROOM_BACKOFF_MAX_MS || 6000;
const ROOM_PROGRESS_THROTTLE_MS = cfg.ROOM_PROGRESS_THROTTLE_MS || 500;

// ----- 色板（design-lock § 7） -----
const GOLD = '#edc22e';
const BROWN = '#5b4a1f';
const PANEL = '#faf8ef';
const INK = '#776e65';
const BTN = '#8f7a66';

const ROOM_CODE_CHARS = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789';

// ===== 工具函数 =====

// ① 超时 race：抖音 tt.request 无原生 timeout，必须用 Promise.race 兜底
function withTimeout(promise, ms) {
  return Promise.race([
    promise,
    new Promise((_, rej) => setTimeout(() => rej(new Error('timeout')), ms)),
  ]);
}

// ② 失败次数 → 退避间隔（取序列上限）
function computeBackoffMs(failCount) {
  const i = Math.max(0, Math.min(failCount, ROOM_BACKOFF_MS.length - 1));
  return ROOM_BACKOFF_MS[i];
}

```

```

function roundRect(ctx, x, y, w, h, r) {
  ctx.beginPath();
  ctx.moveTo(x + r, y);
  ctx.arcTo(x + w, y, x + w, y + h, r);
  ctx.arcTo(x + w, y + h, x, y + h, r);
  ctx.arcTo(x, y + h, x, y, r);
  ctx.arcTo(x, y, x + w, y, r);
  ctx.closePath();
}

function hit(r, t) {
  return t.clientX >= r.x && t.clientX <= r.x + r.w &&
    t.clientY >= r.y && t.clientY <= r.y + r.h;
}

// ----- 签名（复用 RANK_SECRET） -----
function signUidTs(uid, ts) {
  if (!cfg.RANK_SECRET) return "";
  return HMAC.hmacSha256Hex(cfg.RANK_SECRET, String(uid) + '|' + ts);
}
function signScore(uid, score, steps, ts) {
  if (!cfg.RANK_SECRET) return "";
  return HMAC.hmacSha256Hex(
    cfg.RANK_SECRET,
    String(uid) + '|' + Math.floor(Number(score) || 0) + '|' + Math.floor(Number(steps) || 0) + '|' + ts
  );
}

// ----- 网络请求（Promise 化 tt.request） -----
function doRequest(method, url, data) {
  return new Promise((resolve, reject) => {
    if (!isTT || !tt || typeof tt.request !== 'function') {
      reject(new Error('tt.request unavailable'));
      return;
    }
    const opt = {
      url,
      method,
      header: { 'Content-Type': 'application/json' },
      success: (res) => resolve(res && res.data ? res.data : { code: 5, data: null, message: 'empty response' }),
      fail: (e) => reject(e || new Error('request fail')),
    };
    if (method === 'POST') opt.data = data || {};
    tt.request(opt);
  });
}

// ===== RoomClient 工厂 =====
// @param {object} opts
// requestFn: 可选，注入请求函数（用于测试；默认 doRequest）
// beginMatch: (seed) => void 用 seed 初始化本地棋盘（由 game.js 注入）
// exit: () => void 返回主界面（由 game.js 注入）
function createRoomClient(opts) {
  opts = opts || {};
  let requestFn = opts.requestFn || doRequest;
  const api = { beginMatch: opts.beginMatch || null, exit: opts.exit || null };

  let active = false; // 房间流程是否活动（screen==='room'）
  let phase = 'hall'; // 'hall' | 'joinEntry' | 'waiting' | 'playing' | 'result'
  let code = "";
  let uid = "";
  let name = "";

```

```

let amCreator = false; // 是否为建房者 (P1)
let everJoined = false; // 是否已在某房间 (用于区分「新房主等对手」与「终局后重连重开」)
let seed = 0;
let startAt = 0;
let serverStatus = ''; // 服务端 status: waiting/playing/finished/opponent_left/abandoned
let matchResult = 0; // 0 未定 / 1 P1胜 / 2 P2胜 / 3 平 / 4 对手离开 / 5 对手断线
let oppScore = 0;
let oppSteps = 0;
let oppOver = false;
let myScore = 0;
let mySteps = 0;
let lastProgressAt = 0;
let failCount = 0;
let backoffMs = ROOM_BACKOFF_MS[0];
let roomSeq = 0; // ③ 竞态守门：每次进入新房间/重开自增，过期回调丢弃
let pollTimer = null;
let beginTimer = null;
let begunSeed = null; // 当前这一局已用哪个 seed 开局 (防重复 init)
let hasBegunMatch = false; // 是否曾开过局 (用于终局后自动重连重开判定)
let resultSent = false; // 是否已提交终局 result (防重复)
let errorMsg = null;
let joinInput = ''; // 加入房码输入 (最多 6 位)
let lastL = null; // 最近一次 render 的布局 (供 handleTouch 命中判定)

// ----- 轮询 (§ 6 三原则) -----
function stopPolling() {
  if (pollTimer) { clearTimeout(pollTimer); pollTimer = null; }
  if (beginTimer) { clearTimeout(beginTimer); beginTimer = null; }
}
function schedulePoll() {
  if (!active || !code) return;
  const interval = (phase === 'waiting') ? ROOM_WAIT_POLL_MS : Math.max(ROOM_POLL_MS, backoffMs);
  pollTimer = setTimeout(pollLoop, interval);
}
function pollLoop() {
  if (!active || !code) return;
  const seq = ++roomSeq;
  withTimeout(getStateReq(), ROOM_TIMEOUT_RACE_MS)
    .then((resp) => {
      if (seq !== roomSeq || !active || !code) return; // 过期回调丢弃
      failCount = 0;
      backoffMs = ROOM_BACKOFF_MS[0];
      handleState(resp);
      // 仅当仍未终局、房间仍有效时继续轮询 (本地不阻塞)
      if (active && code && matchResult === 0 && serverStatus !== 'finished' && serverStatus !== 'opponent_left' && serverStatus !== 'abandoned') {
        schedulePoll();
      }
    })
    .catch(() => {
      if (seq !== roomSeq || !active || !code) return;
      failCount = Math.min(failCount + 1, ROOM_BACKOFF_MS.length - 1);
      backoffMs = ROOM_BACKOFF_MS[failCount];
      schedulePoll(); // 退避后继续轮询 (绝不阻断本地棋盘)
    });
}

function getStateReq() {
  const ts = Math.floor(Date.now() / 1000);
  const sig = signUidTs(uid, ts);
  const q = 'code=' + encodeURIComponent(code) +
    '&uid=' + encodeURIComponent(uid) +
    '&ts=' + ts +

```

```

    '&sig=' + encodeURIComponent(sig);
    return requestFn('GET', cfg.ROOM_STATE_PATH + '?' + q);
}

// 处理一次 state 轮询结果（design-lock § 4.4 + § 2 状态机）
function handleState(resp) {
    if (!resp || resp.code !== 0) {
        if (resp && resp.code === 3) {
            errorMsg = '房间不存在或已过期';
            stopPolling();
        }
        return;
    }
    serverStatus = resp.data.status || serverStatus;
    seed = (typeof resp.data.seed === 'number') ? resp.data.seed : seed;
    startAt = (typeof resp.data.startAt === 'number') ? resp.data.startAt : startAt;
    matchResult = resp.data.matchResult || 0;
    if (resp.data.opponent) {
        oppScore = resp.data.opponent.score || 0;
        oppSteps = resp.data.opponent.steps || 0;
        oppOver = !resp.data.opponent.over;
    }
    myScore = resp.data.myScore || 0;

    // 终局：切结算卡，停止轮询
    if (matchResult > 0) {
        phase = 'result';
        stopPolling();
        return;
    }
    // 对局中
    if (serverStatus === 'playing') {
        if (phase !== 'playing') {
            phase = 'playing';
            scheduleBegin();
        }
        return; // 保持轮询
    }
    // waiting / 其它：进入等待态
    if (phase === 'waiting') return; // 已在等待
    phase = 'waiting';
    begunSeed = null; // 新一局（可能是终局后 reset）准备用新 seed 重开
    stopPolling();
    if (hasBegunMatch) {
        // 之前已开过局（终局后被 reset）→ 自动重连重开（对手 polling 到此态同理）
        rejoinToRestart();
    } else {
        schedulePoll(); // 新房主等对手
    }
}

// 到达 playing 后，按 startAt 本地时钟精确开局（双方同 startAt → 同步开局）
function scheduleBegin() {
    const now = Date.now();
    const delay = Math.max(0, (startAt || 0) - now);
    if (beginTimer) { clearTimeout(beginTimer); beginTimer = null; }
    beginTimer = setTimeout(() => {
        beginTimer = null;
        if (!active || phase !== 'playing') return;
        if (begunSeed === seed) return; // 本局已开局，防重复 init
        begunSeed = seed;
        hasBegunMatch = true;
    }, delay);
}

```



```
    if (api.beginMatch) api.beginMatch(seed);
  }, delay);
  schedulePoll();
}

// ----- 对外动作 -----
function open(u, n) {
  active = true;
  uid = u || uid;
  name = n || name;
  phase = 'hall';
  code = "";
  seed = 0; startAt = 0; serverStatus = ""; matchResult = 0;
  oppScore = oppSteps = 0; oppOver = false;
  myScore = mySteps = 0;
  failCount = 0; backoffMs = ROOM_BACKOFF_MS[0];
  roomSeq++; // 作废旧回调
  everJoined = false; begunSeed = null; hasBegunMatch = false; resultSent = false;
  errorMsg = null; joinInput = "";
  stopPolling();
}

function exit() {
  // playing/waiting 期礼貌 POST leave；无论如何回到主界面
  const wasInRoom = !!code && (serverStatus === 'playing' || serverStatus === 'waiting' || phase === 'playing' || phase === 'waiting');
  active = false;
  stopPolling();
  if (wasInRoom && isTT) {
    const ts = Math.floor(Date.now() / 1000);
    const sig = signUidTs(uid, ts);
    const req = requestFn('POST', cfg.ROOM_LEAVE_PATH, { code, uid, ts, sig });
    withTimeout(req, ROOM_TIMEOUT_RACE_MS).catch(() => {}); // 失败静默
  }
  code = ""; serverStatus = ""; phase = 'hall';
  if (api.exit) api.exit();
}

function createRoom() {
  if (!uid) return;
  const ts = Math.floor(Date.now() / 1000);
  const sig = signUidTs(uid, ts);
  errorMsg = null;
  withTimeout(requestFn('POST', cfg.ROOM_CREATE_PATH, { uid, name, ts, sig }), ROOM_TIMEOUT_RACE_MS)
    .then((resp) => {
      if (resp && resp.code === 0 && resp.data) {
        code = resp.data.code;
        seed = resp.data.seed;
        amCreator = true;
        everJoined = true;
        phase = 'waiting';
        serverStatus = 'waiting';
        startAt = resp.data.startAt || 0;
        begunSeed = null;
        roomSeq++; // 新房间作废旧回调
        schedulePoll();
      } else {
        errorMsg = (resp && resp.message) || '建房失败';
      }
    })
    .catch(() => { errorMsg = '建房请求失败， 请检查网络'; });
}
```

```

function joinRoom(inputCode) {
  if (!uid) return;
  const c = String(inputCode || '').toUpperCase();
  if (!/^[A-Z0-9]{6}$/.test(c)) { errorMsg = '房码格式为 6 位字母或数字'; return; }
  const ts = Math.floor(Date.now() / 1000);
  const sig = signUidTs(uid, ts);
  errorMsg = null;
  withTimeout(requestFn('POST', cfg.ROOM_JOIN_PATH, { code: c, uid, name, ts, sig }), ROOM_TIMEOUT_RACE_MS)
    .then((resp) => {
      if (resp && resp.code === 0 && resp.data) {
        code = c;
        seed = resp.data.seed;
        serverStatus = resp.data.status || 'waiting';
        const players = resp.data.players || {};
        amCreator = Object.keys(players)[0] === String(uid);
        everJoined = true;
        begunSeed = null;
        if (serverStatus === 'playing') {
          phase = 'playing';
          startAt = resp.data.startAt || 0;
          scheduleBegin();
        } else {
          phase = 'waiting';
          startAt = resp.data.startAt || 0;
          schedulePoll();
        }
      } else if (resp && resp.code === 3) {
        errorMsg = '房间不存在或已过期';
      } else if (resp && resp.code === 4) {
        errorMsg = '房间已满';
      } else {
        errorMsg = (resp && resp.message) || '加入失败';
      }
    })
    .catch(() => { errorMsg = '加入请求失败， 请检查网络'; });
}

```

```

// 再来一局（design-lock § 3 Q5）：reset → waiting；随后自动重连重开（对手 polling 到此态同理）
function requestRestart() {
  if (!code || !uid) return;
  const ts = Math.floor(Date.now() / 1000);
  const sig = signUidTs(uid, ts);
  withTimeout(requestFn('POST', cfg.ROOM_RESET_PATH, { code, uid, ts, sig }), ROOM_TIMEOUT_RACE_MS)
    .then((resp) => {
      if (resp && resp.code === 0) {
        phase = 'waiting';
        serverStatus = 'waiting';
        matchResult = 0;
        begunSeed = null;
        resultSent = false;
        stopPolling();
        rejoinToRestart(); // 自动重连触发重开
      } else if (resp && resp.code === 4) {
        // 对手已先 reset（status 已 waiting），我直接重连重开
        rejoinToRestart();
      } else {
        errorMsg = (resp && resp.message) || '再来一局失败';
      }
    })
    .catch(() => { errorMsg = '再来一局请求失败'; });
}

```

```
// 重连 / 重开：POST join (reconnect) 触发 playing (reset 后双方槽位仍在)
function rejoinToRestart() {
  const ts = Math.floor(Date.now() / 1000);
  const sig = signUidTs(uid, ts);
  withTimeout(requestFn('POST', cfg.ROOM_JOIN_PATH, { code, uid, name, ts, sig }, ROOM_TIMEOUT_RACE_MS)
    .then((resp) => {
      if (resp && resp.code === 0) {
        seed = (typeof resp.data.seed === 'number') ? resp.data.seed : seed;
        serverStatus = resp.data.status || serverStatus;
        if (serverStatus === 'playing' && phase !== 'playing') {
          phase = 'playing';
          startAt = resp.data.startAt || 0;
          scheduleBegin();
        }
        schedulePoll();
      } else {
        errorMsg = (resp && resp.message) || '重连失败';
      }
    })
    .catch(() => { errorMsg = '重连失败，请重试'; }));
}
```

```
// 进度上报（§ 4 Q4：≥500ms 或每步取先到者；仅展示层，不回放棋局）
function reportProgress(score, steps, over) {
  if (!active || phase !== 'playing' || !code) return;
  myScore = score; mySteps = steps;
  const now = Date.now();
  if (over) { sendProgress(score, steps, over); return; } // 终局立即上报
  if (now - lastProgressAt < ROOM_PROGRESS_THROTTLE_MS) return; // 节流
  sendProgress(score, steps, over);
}
function sendProgress(score, steps, over) {
  lastProgressAt = Date.now();
  const ts = Math.floor(Date.now() / 1000);
  const sig = signScore(uid, score, steps, ts);
  const req = requestFn('POST', cfg.ROOM_PROGRESS_PATH, { code, uid, score, steps, over, ts, sig });
  withTimeout(req, ROOM_TIMEOUT_RACE_MS).catch(() => {}); // 失败静默，本地不阻塞
}
```

```
// 提交终局结果（§ 4.5）：满格(over) won=false；先到 2048 won=true。由轮询 matchResult 驱动结算卡。
function submitMyResult(score, steps, won) {
  if (!active || !code || resultSent) return;
  resultSent = true;
  const ts = Math.floor(Date.now() / 1000);
  const sig = signScore(uid, score, steps, ts);
  const req = requestFn('POST', cfg.ROOM_RESULT_PATH, { code, uid, score, steps, won, ts, sig });
  withTimeout(req, ROOM_TIMEOUT_RACE_MS).catch(() => {});
}
```

```
// ----- 触摸（UI 按钮命中；棋盘滑动由 game.js 处理） -----
function handleTouch(sx, sy, t) {
  if (!active || !lastL) return false;
  const L = lastL;
  if (phase === 'hall') {
    if (hit(hallBackRect(L), t)) { exit(); return true; }
    if (hit(hallCreateRect(L), t)) { createRoom(); return true; }
    if (hit(hallJoinRect(L), t)) { phase = 'joinEntry'; joinInput = ''; return true; }
    return false;
  }
  if (phase === 'joinEntry') {
    if (hit(joinBackRect(L), t)) { phase = 'hall'; return true; }
    if (hit(joinBackspaceRect(L), t)) { joinInput = joinInput.slice(0, -1); return true; }
  }
}
```

```

    if (joinInput.length === 6 && hit(joinConfirmRect(L), t)) { joinRoom(joinInput); return true; }
    const keys = joinKeyRects(L);
    for (const k of keys) {
        if (hit(k, t)) {
            if (joinInput.length < 6) joinInput += k.ch;
            return true;
        }
    }
    return false;
}
if (phase === 'waiting') {
    if (hit(waitingBackRect(L), t)) { exit(); return true; }
    return false;
}
if (phase === 'result') {
    if (hit(resultAgainRect(L), t)) { requestRestart(); return true; }
    if (hit(resultBackRect2(L), t)) { exit(); return true; }
    return false;
}
// playing 期无 UI 按钮（滑动交给 game.js），不消费
return false;
}

// ----- 渲染（game.js 在 draw() 末尾调用，screen==='room' 时） -----
function render(ctx, L) {
    if (!active) return;
    lastL = L;
    if (phase === 'playing') { drawHUD(ctx, L); return; } // 对战期仅画 HUD，棋盘由 game.js 画
    // hall / joinEntry / waiting / result：全屏覆盖
    ctx.fillStyle = 'rgba(0,0,0,0.72)';
    ctx.fillRect(0, 0, L.W, L.H);
    if (phase === 'hall') drawHall(ctx, L);
    else if (phase === 'joinEntry') drawJoinEntry(ctx, L);
    else if (phase === 'waiting') drawWaiting(ctx, L);
    else if (phase === 'result') drawResult(ctx, L);
}

// ===== Canvas UI（单一数据源：rect 同时供绘制与命中判定） =====
function panelRect(L) { return { x: L.PAD, y: 56, w: L.W - L.PAD * 2, h: L.H - 56 - 24 }; }

function hallBackRect(L) { const p = panelRect(L); return { x: p.x + 10, y: p.y + 10, w: 28, h: 28 }; }
function hallCreateRect(L) { const p = panelRect(L); return { x: p.x + 16, y: p.y + p.h - 120, w: (p.w - 16 * 3) / 2, h: 44 }; }
function hallJoinRect(L) { const p = panelRect(L); return { x: p.x + 16 * 2 + (p.w - 16 * 3) / 2, y: p.y + p.h - 120, w: (p.w - 16 * 3) / 2, h: 44 }; }

function joinBackRect(L) { const p = panelRect(L); return { x: p.x + 10, y: p.y + 10, w: 28, h: 28 }; }
function joinKeyRects(L) {
    const p = panelRect(L);
    const cols = 6, rows = 6, pad = 8;
    const areaX = p.x + 16;
    const areaY = p.y + 100;
    const areaW = p.w - 32;
    const kw = (areaW - pad * (cols - 1)) / cols;
    const kh = 36;
    const rects = [];
    for (let i = 0; i < 36; i++) {
        const r = Math.floor(i / cols), c = i % cols;
        rects.push({ ch: ROOM_CODE_CHARS[i], x: areaX + c * (kw + pad), y: areaY + r * (kh + pad), w: kw, h: kh });
    }
    return rects;
}
function joinBackspaceRect(L) {
    const p = panelRect(L); const ks = joinKeyRects(L); const last = ks[ks.length - 1];

```

```

    return { x: p.x + 16, y: last.y + last.h + 10, w: (p.w - 32) * 0.5 - 4, h: 40 };
}
function joinConfirmRect(L) {
    const p = panelRect(L); const ks = joinKeyRects(L); const last = ks[ks.length - 1];
    return { x: p.x + 16 + (p.w - 32) * 0.5 + 4, y: last.y + last.h + 10, w: (p.w - 32) * 0.5 - 4, h: 40 };
}

function waitingBackRect(L) { const p = panelRect(L); return { x: p.x + 10, y: p.y + 10, w: 28, h: 28 }; }
function resultBackRect(L) { const p = panelRect(L); return { x: p.x + 10, y: p.y + 10, w: 28, h: 28 }; }
function resultAgainRect(L) { const p = panelRect(L); return { x: p.x + 16, y: p.y + p.h - 120, w: (p.w - 16 * 3) / 2, h: 44 }; }
function resultBackRect2(L) { const p = panelRect(L); return { x: p.x + 16 * 2 + (p.w - 16 * 3) / 2, y: p.y + p.h - 120, w: (p.w - 16 * 3) / 2, h: 44 }; }

function drawPanel(ctx, L) {
    const p = panelRect(L);
    ctx.save();
    ctx.shadowColor = 'rgba(0,0,0,0.3)'; ctx.shadowBlur = 20; ctx.shadowOffsetY = 6;
    ctx.fillStyle = PANEL; roundRect(ctx, p.x, p.y, p.w, p.h, 16); ctx.fill();
    ctx.restore();
    return p;
}
function drawCloseX(ctx, r) {
    ctx.strokeStyle = '#bbada0'; ctx.lineWidth = 2;
    ctx.beginPath();
    ctx.moveTo(r.x + 8, r.y + 8); ctx.lineTo(r.x + r.w - 8, r.y + r.h - 8);
    ctx.moveTo(r.x + r.w - 8, r.y + 8); ctx.lineTo(r.x + 8, r.y + r.h - 8);
    ctx.stroke();
}

function drawHall(ctx, L) {
    const p = drawPanel(ctx, L);
    drawCloseX(ctx, hallBackRect(L));
    ctx.fillStyle = INK; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
    ctx.font = 'bold 20px sans-serif';
    ctx.fillText('好友对战 · 房间', L.W / 2, p.y + 30);
    ctx.fillStyle = BTN; ctx.font = '12px sans-serif';
    ctx.fillText('系统返回键 / × 退出', L.W / 2, p.y + 48);
    ctx.fillStyle = INK; ctx.font = '14px sans-serif';
    ctx.fillText('创建房间分享房码，或输入房码加入', L.W / 2, p.y + 86);
    if (errorMsg) { ctx.fillStyle = '#c0392b'; ctx.font = '14px sans-serif'; ctx.fillText(errorMsg, L.W / 2, p.y + 112); }
    const cb = hallCreateRect(L);
    ctx.fillStyle = GOLD; roundRect(ctx, cb.x, cb.y, cb.w, cb.h, cb.h / 2); ctx.fill();
    ctx.fillStyle = BROWN; ctx.font = 'bold 16px sans-serif'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
    ctx.fillText('创建房间', cb.x + cb.w / 2, cb.y + cb.h / 2);
    const jb = hallJoinRect(L);
    ctx.fillStyle = BTN; roundRect(ctx, jb.x, jb.y, jb.w, jb.h, jb.h / 2); ctx.fill();
    ctx.fillStyle = '#fff'; ctx.fillText('加入房间', jb.x + jb.w / 2, jb.y + jb.h / 2);
}

function drawJoinEntry(ctx, L) {
    const p = drawPanel(ctx, L);
    drawCloseX(ctx, joinBackRect(L));
    ctx.fillStyle = INK; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
    ctx.font = 'bold 18px sans-serif';
    ctx.fillText('输入房码加入', L.W / 2, p.y + 26);
    // 已输入房码 (6 格)
    const dispY = p.y + 52; const cellW = (p.w - 32) / 6;
    for (let i = 0; i < 6; i++) {
        const x = p.x + 16 + i * cellW;
        ctx.fillStyle = '#cdc1b4'; roundRect(ctx, x, dispY, cellW - 6, 40, 8); ctx.fill();
        const ch = joinInput[i] || '';
        ctx.fillStyle = ch ? BROWN : '#bbada0'; ctx.font = 'bold 22px sans-serif'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
        ctx.fillText(ch, x + (cellW - 6) / 2, dispY + 20);
    }
}

```

```

}
// 键盘 6×6
const keys = joinKeyRects(L);
for (const k of keys) {
  ctx.fillStyle = '#eee4da'; roundRect(ctx, k.x, k.y, k.w, k.h, 8); ctx.fill();
  ctx.fillStyle = BROWN; ctx.font = 'bold 18px sans-serif'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
  ctx.fillText(k.ch, k.x + k.w / 2, k.y + k.h / 2);
}
// 退格 + 加入
const bs = joinBackspaceRect(L);
ctx.fillStyle = BTN; roundRect(ctx, bs.x, bs.y, bs.w, bs.h, bs.h / 2); ctx.fill();
ctx.fillStyle = '#fff'; ctx.font = 'bold 14px sans-serif'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
ctx.fillText(' ', bs.x + bs.w / 2, bs.y + bs.h / 2);
const jb = joinConfirmRect(L);
ctx.fillStyle = joinInput.length === 6 ? GOLD : '#cdc1b4';
roundRect(ctx, jb.x, jb.y, jb.w, jb.h, jb.h / 2); ctx.fill();
ctx.fillStyle = joinInput.length === 6 ? BROWN : '#fff';
ctx.font = 'bold 16px sans-serif'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
ctx.fillText('加入', jb.x + jb.w / 2, jb.y + jb.h / 2);
if (errorMsg) { ctx.fillStyle = '#c0392b'; ctx.font = '13px sans-serif'; ctx.textAlign = 'center'; ctx.fillText(errorMsg, L.W / 2, jb.y + jb.h + 16); }
}

```

```

function drawWaiting(ctx, L) {
  const p = drawPanel(ctx, L);
  drawCloseX(ctx, waitingBackRect(L));
  ctx.fillStyle = INK; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
  ctx.font = 'bold 20px sans-serif';
  ctx.fillText('房间已创建', L.W / 2, p.y + 30);
  ctx.fillStyle = BTN; ctx.font = '12px sans-serif';
  ctx.fillText('系统返回键 / × 退出', L.W / 2, p.y + 48);
  ctx.fillStyle = BROWN; ctx.font = 'bold 40px sans-serif';
  ctx.fillText(code, L.W / 2, p.y + 108);
  ctx.fillStyle = INK; ctx.font = '14px sans-serif';
  ctx.fillText('把房码分享给好友，等待对手加入...', L.W / 2, p.y + 148);
  if (errorMsg) { ctx.fillStyle = '#c0392b'; ctx.font = '14px sans-serif'; ctx.fillText(errorMsg, L.W / 2, p.y + 178); }
}

```

```

function drawHUD(ctx, L) {
  // 右上：对手信息（§ 7 Q7：仅分数 + 步数小字，不渲染缩略棋盘）
  ctx.textAlign = 'right'; ctx.textBaseline = 'middle';
  ctx.fillStyle = INK; ctx.font = 'bold 13px sans-serif';
  const oppLabel = amCreator ? '对手: P2' : '对手: P1';
  ctx.fillText(oppLabel + ' ' + oppScore + ' · ' + oppSteps + '步', L.W - 12, 30);
  // 顶部中间：比分条
  const bw = L.W - 24, bx = 12, by = 44;
  ctx.fillStyle = '#bbada0'; roundRect(ctx, bx, by, bw, 6, 3); ctx.fill();
  const total = Math.max(1, myScore + oppScore);
  const myW = bw * (myScore / total);
  ctx.fillStyle = GOLD; roundRect(ctx, bx, by, myW, 6, 3); ctx.fill();
  ctx.fillStyle = INK; ctx.font = 'bold 12px sans-serif'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
  ctx.fillText('比分 ' + myScore + ' vs ' + oppScore, L.W / 2, by + 22);
  // 底部：我方信息 + 同步状态（失败 failCount>0 显示「同步中…」）
  ctx.fillStyle = BTN; ctx.font = 'bold 13px sans-serif'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
  const sync = (failCount > 0) ? '同步中...' : '对战中';
  const endHint = (state && state.over) ? '已终局，等待结算...' : '';
  ctx.fillText('我: ' + myScore + ' · ' + mySteps + '步 ' + sync + endHint, L.W / 2, L.H - 16);
}

```

```

function resultText() {
  const iWin = (matchResult === 1 && amCreator) || (matchResult === 2 && !amCreator) || matchResult === 4 || matchResult === 5;
  const iLose = (matchResult === 1 && !amCreator) || (matchResult === 2 && amCreator);
  if (matchResult === 3) return { title: '平局', sub: '比分相同，势均力敌', color: '#8f7a66' };
}

```

```
// ===== 文件：server/store.js （程序起始 / 第 1 个源文件） =====
// server/store.js
// 排行榜 + 天梯存储（纯逻辑，可单测）。零新增 npm 依赖。
//
// △ 已支持「持久化」：按环境变量 RANK_STORE 选择后端：
// - 'memory'（默认）：进程内 Map，重启即清空，仅本地/测试用
// - 'file'：JSON 文件落盘（自托管 VPS / 普通 Node 服务可直接持久化）
// - 'upstash'：Upstash Redis REST（serverless 如 Vercel / Cloudflare 推荐，零原生依赖）
//
// 排行榜接口（三个后端完全一致，便于切换、无需改 index.js / 前端）：
// recordScore(uid, name, score) —— 记录玩家最高分（同名 uid 只保留最高，异步）
// getRankView(uid, limit) —— 返回 { top, selfRank, selfName, selfScore }（异步）
//
// 天梯接口（Phase 1 异步匹配，三个后端一致）：
// saveSnapshot(s) —— 写本人天梯快照 {uid,name,score,steps,boardSummary,ts,synthetic}
// matchSnapshot(score, uid, band) —— 按分数带宽取候选（排除本人、排除 24h 内已匹配 uid），返回一条或 null
// pushHistory(uid, rec) —— 写战绩 LPUSH + LTRIM 保留最近 50
// getHistory(uid, limit) —— 读历史，返回按 ts 倒序数组
//
// 接口：
// recordScore(uid, name, score) —— 记录某个玩家的最高分（同名 uid 只保留最高）
// getRankView(uid, limit) —— 返回 { top, selfRank, selfName, selfScore }
// saveSnapshot / matchSnapshot / pushHistory / getHistory —— 天梯相关（见上）

const fs = require('fs');
const path = require('path');

// 房间 key 过期时间（600s，对应重连窗口 / TTL；design-lock § 1.1 ROOM_TTL_SECONDS）
const ROOM_TTL_MS = 600000;

// ----- 通用层：把后端提供的「条目集合」包装成统一接口 -----
// backend 需实现：
// async getEntries() -> [{ uid, name, score }]
// async upsertMax(uid, name, s) -> 仅当 s 大于已有分时才写入
// 天梯需额外实现（memory/file/upstash 三后端一致）：
// async ladderSaveSnapshot(s)
// async ladderGetSnapshotsInRange(lo, hi) -> [snapshot]
// async ladderSetLastOpp(uid, oppUid, ts)
// async ladderGetLastOpp(uid) -> {uid, ts} | null
// async ladderPushHistory(uid, rec) (LPUSH + LTRIM 50)
// async ladderGetHistory(uid, limit) -> [rec] (newest first)
function makeStore(backend) {
  async function recordScore(uid, name, score) {
    if (!uid) return false;
    const sc = Math.max(0, Math.floor(Number(score) || 0));
    await backend.upsertMax(String(uid), String(name || '玩家').slice(0, 16), sc);
    return true;
  }

  async function getRankView(uid, limit) {
    limit = limit || 100;
    const all = (await backend.getEntries()).map((p) => ({
      uid: p.uid, name: p.name, score: p.score,
    }));
    // 分数降序；同分按 uid 字典序（稳定、确定）
    all.sort((a, b) => b.score - a.score || (a.uid < b.uid ? -1 : a.uid > b.uid ? 1 : 0));
    const ranked = all.map((p, i) => ({
      uid: p.uid, name: p.name, score: p.score, rank: i + 1, isSelf: p.uid === uid,
    }));
    const top = ranked.slice(0, limit).map((x) => ({
      rank: x.rank, name: x.name, score: x.score, isSelf: x.isSelf,
    }));
  }
}
```

```
let selfRank = ranked.length;
let selfScore = 0;
let selfName = '';
const me = ranked.find((x) => x.uid === uid);
if (me) { selfRank = me.rank; selfScore = me.score; selfName = me.name; }
return { top, selfRank, selfName, selfScore };
}

// ----- 天梯方法 -----
async function saveSnapshot(s) {
  if (!s || !s.uid) return false;
  await backend.ladderSaveSnapshot(s);
  return true;
}

// 在快照池按分数带宽匹配对手：排除本人、排除 24h 内已匹配的 uid,
// 取分数最接近的一条；无候选返回 null（调用方应降级为合成对手）。
async function matchSnapshot(score, uid, band) {
  const sc = Math.floor(Number(score) || 0);
  const b = Math.max(0, Math.floor(Number(band) || 0));
  const lo = Math.max(0, sc - b);
  const hi = sc + b;
  const last = await backend.ladderGetLastOpp(uid);
  const now = Math.floor(Date.now() / 1000);
  const exclude = new Set([String(uid)]);
  if (last && Number(last.ts) && (now - Number(last.ts)) < 24 * 3600) {
    exclude.add(String(last.uid));
  }
  const cands = (await backend.ladderGetSnapshotsInRange(lo, hi))
    .filter((s) => s && s.uid && !exclude.has(String(s.uid)));
  if (!cands.length) return null;
  cands.sort((a, b) => Math.abs(Number(a.score) - sc) - Math.abs(Number(b.score) - sc));
  const pick = cands[0];
  // 记录本次匹配对手，用于 24h 内去重
  await backend.ladderSetLastOpp(uid, pick.uid, now);
  return pick;
}

async function pushHistory(uid, rec) {
  if (!uid) return false;
  await backend.ladderPushHistory(String(uid), rec);
  return true;
}

async function getHistory(uid, limit) {
  return backend.ladderGetHistory(String(uid), limit || 50);
}

// ----- 房间方法（Phase 2 实时对战，三后端一致） -----
// 这些方法仅做「委托」：真实存储由 backend.xxx 实现（memory/file/upstash 各一份）。
async function roomSet(code, room) {
  await backend.roomSet(String(code), room);
  return true;
}

async function roomGet(code) {
  return backend.roomGet(String(code)); // 不存在返回 null
}

async function roomProgress(code, uid, p) {
  await backend.roomProgress(String(code), String(uid), p);
  return true;
}

async function roomResult(code, uid, r) {
```



```

    await backend.roomResult(String(code), String(uid), r);
    return true;
  }
  async function roomTouch(code) {
    await backend.roomTouch(String(code));
    return true;
  }

  return {
    recordScore,
    getRankView,
    saveSnapshot,
    matchSnapshot,
    pushHistory,
    getHistory,
    roomSet,
    roomGet,
    roomProgress,
    roomResult,
    roomTouch,
    _dump: () => backend.getEntries(),
    _backend: backend.type,
  };
}

// ----- 内存后端 -----
function createMemoryStore() {
  const players = new Map(); // uid -> { name, score }
  // 天梯状态（进程内）
  const ladderSnaps = new Map(); // member(uid:ts) -> snapshot
  const ladderLastOpp = new Map(); // uid -> { uid, ts }
  const ladderHist = new Map(); // uid -> [rec,...] (newest first, ≤50)
  const roomMap = new Map(); // code -> { obj, expireAt }

  function normalizeSnap(s) {
    return {
      uid: String(s.uid),
      name: String(s.name || '玩家').slice(0, 16),
      score: Math.floor(Number(s.score) || 0),
      steps: Math.floor(Number(s.steps) || 0),
      boardSummary: s.boardSummary == null ? null : s.boardSummary,
      ts: Number(s.ts),
      synthetic: !!s.synthetic,
    };
  }

  return makeStore({
    type: 'memory',
    async getEntries() {
      return Array.from(players.entries()).map(([uid, v]) => ({ uid, name: v.name, score: v.score }));
    },
    async upsertMax(uid, name, s) {
      const cur = players.get(uid);
      if (!cur || s > cur.score) players.set(uid, { name, score: s });
    },
    // 天梯
    async ladderSaveSnapshot(s) {
      const n = normalizeSnap(s);
      ladderSnaps.set(String(n.uid) + '.' + n.ts, n);
    },
    async ladderGetSnapshotsInRange(lo, hi) {
      const out = [];

```

```

    for (const s of ladderSnaps.values()) {
      if (s.score >= lo && s.score <= hi) out.push(s);
    }
    return out;
  },
  async ladderSetLastOpp(uid, oppUid, ts) {
    ladderLastOpp.set(String(uid), { uid: String(oppUid), ts: Number(ts) });
  },
  async ladderGetLastOpp(uid) {
    return ladderLastOpp.get(String(uid)) || null;
  },
  async ladderPushHistory(uid, rec) {
    const arr = ladderHist.get(String(uid)) || [];
    arr.unshift(rec);
    if (arr.length > 50) arr.length = 50; // 仅保留最近 50
    ladderHist.set(String(uid), arr);
  },
  async ladderGetHistory(uid, limit) {
    const arr = ladderHist.get(String(uid)) || [];
    return arr.slice(0, Math.max(1, limit || 50));
  },
  // 房间（Phase 2）：整体存 { obj, expireAt }，读时检查过期
  async roomSet(code, room) {
    roomMap.set(String(code), { obj: room, expireAt: Date.now() + ROOM_TTL_MS });
  },
  async roomGet(code) {
    const e = roomMap.get(String(code));
    if (!e) return null;
    if (Date.now() > e.expireAt) { roomMap.delete(String(code)); return null; }
    return e.obj;
  },
  async roomProgress(code, uid, p) {
    const e = roomMap.get(String(code));
    if (!e) return;
    e.obj.players = e.obj.players || {};
    const player = e.obj.players[String(uid)] || { uid: String(uid) };
    player.score = Math.floor(Number(p.score) || 0);
    player.steps = Math.floor(Number(p.steps) || 0);
    player.over = !!p.over;
    player.updatedAt = Date.now();
    e.obj.players[String(uid)] = player;
    e.expireAt = Date.now() + ROOM_TTL_MS;
  },
  async roomResult(code, uid, r) {
    const e = roomMap.get(String(code));
    if (!e) return;
    e.obj.results = e.obj.results || {};
    e.obj.results[String(uid)] = r;
    e.expireAt = Date.now() + ROOM_TTL_MS;
  },
  async roomTouch(code) {
    const e = roomMap.get(String(code));
    if (e) e.expireAt = Date.now() + ROOM_TTL_MS;
  },
});
}

// ----- 文件后端（JSON 落盘） -----
function createFileStore(file) {
  const fp = file || process.env.RANK_FILE || path.join(__dirname, 'rank-data.json');
  const ladderFp = path.join(path.dirname(fp), 'ladder-data.json');
  let players = new Map();

```

```
// 天梯状态
const ladderSnaps = new Map();
const ladderLastOpp = new Map();
const ladderHist = new Map();

const roomDir = path.dirname(fp);
function roomFilePath(code) { return path.join(roomDir, 'room-' + String(code) + '.json'); }
function fileRoomGet(code) {
  try {
    const raw = fs.readFileSync(roomFilePath(code), 'utf8');
    const o = JSON.parse(raw);
    if (o && o.expireAt && Date.now() > o.expireAt) {
      try { fs.unlinkSync(roomFilePath(code)); } catch (e2) { /* 忽略 */ }
      return null;
    }
    return o ? o.obj : null;
  } catch (e) { return null; }
}
function fileRoomSet(code, room) {
  try {
    fs.writeFileSync(roomFilePath(code), JSON.stringify({ obj: room, expireAt: Date.now() + ROOM_TTL_MS }));
  } catch (e) { /* 忽略写失败 */ }
}

try {
  const raw = fs.readFileSync(fp, 'utf8');
  const arr = JSON.parse(raw);
  if (Array.isArray(arr)) {
    for (const it of arr) players.set(it.uid, { name: it.name, score: it.score });
  }
} catch (e) { /* 文件不存在或损坏则从头开始 */ }

// 载入天梯数据 (best-effort)
try {
  const rawL = fs.readFileSync(ladderFp, 'utf8');
  const o = JSON.parse(rawL) || {};
  if (o.snaps) for (const [k, v] of Object.entries(o.snaps)) ladderSnaps.set(k, v);
  if (o.lastOpp) for (const [k, v] of Object.entries(o.lastOpp)) ladderLastOpp.set(k, v);
  if (o.hist) for (const [k, v] of Object.entries(o.hist)) ladderHist.set(k, v);
} catch (e) { /* 忽略加载失败 */ }

function persist() {
  const arr = Array.from(players.entries()).map(([uid, v]) => ({ uid, name: v.name, score: v.score }));
  try { fs.writeFileSync(fp, JSON.stringify(arr, null, 0)); } catch (e) { /* 忽略写失败 */ }
}

function persistLadder() {
  try {
    const o = {
      snaps: Object.fromEntries(ladderSnaps),
      lastOpp: Object.fromEntries(ladderLastOpp),
      hist: Object.fromEntries(ladderHist),
    };
    fs.writeFileSync(ladderFp, JSON.stringify(o, null, 0));
  } catch (e) { /* 忽略写失败 (best-effort) */ }
}

function normalizeSnap(s) {
  return {
    uid: String(s.uid),
    name: String(s.name || '玩家').slice(0, 16),
    score: Math.floor(Number(s.score) || 0),
  }
}
```

```

    steps: Math.floor(Number(s.steps) || 0),
    boardSummary: s.boardSummary == null ? null : s.boardSummary,
    ts: Number(s.ts),
    synthetic: !!s.synthetic,
  };
}

return makeStore({
  type: 'file',
  async getEntries() {
    return Array.from(players.entries()).map(([uid, v]) => ({ uid, name: v.name, score: v.score }));
  },
  async upsertMax(uid, name, s) {
    const cur = players.get(uid);
    if (!cur || s > cur.score) { players.set(uid, { name, score: s }); persist(); }
  },
  // 天梯
  async ladderSaveSnapshot(s) {
    const n = normalizeSnap(s);
    ladderSnaps.set(String(n.uid) + '.' + n.ts, n);
    persistLadder();
  },
  async ladderGetSnapshotsInRange(lo, hi) {
    const out = [];
    for (const s of ladderSnaps.values()) {
      if (s.score >= lo && s.score <= hi) out.push(s);
    }
    return out;
  },
  async ladderSetLastOpp(uid, oppUid, ts) {
    ladderLastOpp.set(String(uid), { uid: String(oppUid), ts: Number(ts) });
    persistLadder();
  },
  async ladderGetLastOpp(uid) {
    return ladderLastOpp.get(String(uid)) || null;
  },
  async ladderPushHistory(uid, rec) {
    const arr = ladderHist.get(String(uid)) || [];
    arr.unshift(rec);
    if (arr.length > 50) arr.length = 50;
    ladderHist.set(String(uid), arr);
    persistLadder();
  },
  async ladderGetHistory(uid, limit) {
    const arr = ladderHist.get(String(uid)) || [];
    return arr.slice(0, Math.max(1, limit || 50));
  },
  // 房间（Phase 2）：以 room-<code>.json 落盘（文件名避开 Windows 非法的 ':')
  async roomSet(code, room) { fileRoomSet(code, room); },
  async roomGet(code) { return fileRoomGet(code); },
  async roomProgress(code, uid, p) {
    const room = fileRoomGet(code);
    if (!room) return;
    room.players = room.players || {};
    const player = room.players[String(uid)] || { uid: String(uid) };
    player.score = Math.floor(Number(p.score) || 0);
    player.steps = Math.floor(Number(p.steps) || 0);
    player.over = !!p.over;
    player.updatedAt = Date.now();
    room.players[String(uid)] = player;
    fileRoomSet(code, room);
  },

```

```

    async roomResult(code, uid, r) {
      const room = fileRoomGet(code);
      if (!room) return;
      room.results = room.results || {};
      room.results[String(uid)] = r;
      fileRoomSet(code, room);
    },
    async roomTouch(code) {
      const room = fileRoomGet(code);
      if (room) fileRoomSet(code, room); // 重写即刷新 expireAt
    },
  });
}

// ----- Upstash Redis 后端（serverless） -----
// 排行榜数据模型：有序集合 board(uid=成员, score=分数) + 哈希 meta(uid=字段, name=值)
// 天梯数据模型：
// ladder:snapshots      ZSET score=分值, member=uid:ts
// ladder:snap          HASH member(uid:ts) -> json 快照
// ladder:history:<uid>   LIST 战绩 (LPUSH + LTRIM 50)
// ladder:lastopp:<uid>   HASH {uid, ts} 最近对手 (24h 去重)
function createUpstashStore() {
  const url = (process.env.UPSTASH_REDIS_REST_URL || "").replace(/\/$/, "");
  const token = process.env.UPSTASH_REDIS_REST_TOKEN || "";
  if (!url || !token) throw new Error('使用 upstash 存储需设置 UPSTASH_REDIS_REST_URL 与 UPSTASH_REDIS_REST_TOKEN');
  const BOARD = 'rank:board';
  const META = 'rank:meta';

  // 提交单条 Redis 命令。
  // ⚠ 关键修复：Upstash REST API 的 POST body 必须是「裸 JSON 数组」["COMMAND", "ARG1", ...]
  // （官方文档与 @upstash/redis SDK 均使用此格式）。此前误用 { command: [...] } 包裹对象，
  // Upstash 解析 body 时期望数组却得到对象，返回 "expected JSON array"，导致所有命令失败
  // （前端表现为排行榜 / 天梯战绩 / 房间全部异常）。
  // 所有参数统一转成字符串，避免数字参数（如 LRANGE 的 stop、ZREVRANGE 的 0/-1）被误解析。
  // URL 仅用 base（UPSTASH_REDIS_REST_URL），不在路径里拼接命令，从而彻底规避
  // 「命令拼在 URL 路径里导致的 400 input length too long」（房间 JSON / boardSummary 等大值写入时易触顶）。
  async function rcmd(...args) {
    const command = args.map((a) => (a == null ? '' : String(a)));
    const res = await fetch(url, {
      method: 'POST',
      headers: {
        Authorization: 'Bearer ' + token,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(command),
    });
    const json = await res.json();
    if (json.error) throw new Error('upstash: ' + json.error);
    return json.result;
  }

  function flatToObj(flat) {
    const o = {};
    for (let i = 0; i + 1 < flat.length; i += 2) o[flat[i]] = flat[i + 1];
    return o;
  }

  // 房间（Phase 2）：key = room:<code>，整体 JSON，EXPIRE 600s
  async function upstashRoomGet(code) {
    try {
      const r = await rcmd('GET', 'room:' + String(code));
      if (r == null) return null;
    }
  }

```

```

    const s = typeof r === 'string' ? r : String(r);
    try { return JSON.parse(s); } catch (e) { return null; }
  } catch (e) { return null; } // upstash 不可达：返回 null （不抛）
}
async function upstashRoomSet(code, room) {
  try { await rcmd('SET', 'room:' + String(code), JSON.stringify(room), 'EX', '600'); } catch (e) { /* 静默失败 */ }
}

return makeStore({
  type: 'upstash',
  async getEntries() {
    const flat = (await rcmd('ZREVRANGE', BOARD, 0, -1, 'WITHSCORES')) || [];
    const meta = flatToObj((await rcmd('HGETALL', META)) || []);
    const out = [];
    for (let i = 0; i + 1 < flat.length; i += 2) {
      const uid = flat[i];
      out.push({ uid, name: meta[uid] || '玩家', score: Number(flat[i + 1]) || 0 });
    }
    return out;
  },
  async upsertMax(uid, name, s) {
    await rcmd('ZADD', BOARD, 'GT', String(s), uid); // GT：仅当新分更高才更新
    await rcmd('HSET', META, uid, name);
  },
  // 天梯
  async ladderSaveSnapshot(s) {
    const member = String(s.uid) + '!' + Number(s.ts);
    const snap = {
      uid: String(s.uid),
      name: String(s.name || '玩家').slice(0, 16),
      score: Math.floor(Number(s.score) || 0),
      steps: Math.floor(Number(s.steps) || 0),
      boardSummary: s.boardSummary == null ? null : s.boardSummary,
      ts: Number(s.ts),
      synthetic: !!s.synthetic,
    };
    await rcmd('ZADD', 'ladder:snapshots', String(snap.score), member);
    await rcmd('HSET', 'ladder:snap', member, JSON.stringify(snap));
  },
  async ladderGetSnapshotsInRange(lo, hi) {
    const members = (await rcmd('ZRANGEBYSCORE', 'ladder:snapshots', String(lo), String(hi))) || [];
    const all = flatToObj((await rcmd('HGETALL', 'ladder:snap')) || []);
    const out = [];
    for (const m of members) {
      if (all[m]) {
        try { out.push(JSON.parse(all[m])); } catch (e) { /* 忽略损坏项 */ }
      }
    }
    return out;
  },
  async ladderSetLastOpp(uid, oppUid, ts) {
    await rcmd('HSET', 'ladder:lastopp:' + String(uid), 'uid', String(oppUid), 'ts', String(ts));
  },
  async ladderGetLastOpp(uid) {
    const o = flatToObj((await rcmd('HGETALL', 'ladder:lastopp:' + String(uid))) || []);
    if (!o.uid) return null;
    return { uid: o.uid, ts: Number(o.ts) || 0 };
  },
  async ladderPushHistory(uid, rec) {
    await rcmd('LPUSH', 'ladder:history:' + String(uid), JSON.stringify(rec));
    await rcmd('LTRIM', 'ladder:history:' + String(uid), '0', '49'); // 保留最近 50
  },
});

```

```

async ladderGetHistory(uid, limit) {
  try {
    const arr = (await rcmd('LRANGE', 'ladder:history:' + String(uid), '0', String(Math.max(0, (limit || 50) - 1)))) || [];
    return arr.map((s) => { try { return JSON.parse(s); } catch (e) { return null; } }).filter(Boolean);
  } catch (e) {
    // upstash 不可达 / 出错：降级返回空历史，避免 getHistory 抛异常使前端天梯战绩卡死
    return [];
  }
},
// 房间 (Phase 2)
async roomSet(code, room) { await upstashRoomSet(code, room); },
async roomGet(code) { return upstashRoomGet(code); },
async roomProgress(code, uid, p) {
  try {
    const room = await upstashRoomGet(code);
    if (!room) return;
    room.players = room.players || {};
    const player = room.players[String(uid)] || { uid: String(uid) };
    player.score = Math.floor(Number(p.score) || 0);
    player.steps = Math.floor(Number(p.steps) || 0);
    player.over = !!p.over;
    player.updatedAt = Date.now();
    room.players[String(uid)] = player;
    await upstashRoomSet(code, room);
  } catch (e) { /* 静默失败 */ }
},
async roomResult(code, uid, r) {
  try {
    const room = await upstashRoomGet(code);
    if (!room) return;
    room.results = room.results || {};
    room.results[String(uid)] = r;
    await upstashRoomSet(code, room);
  } catch (e) { /* 静默失败 */ }
},
async roomTouch(code) {
  try { await rcmd('EXPIRE', 'room:' + String(code), '600'); } catch (e) { /* 静默失败 */ }
},
});
}

// ----- 工厂：按环境变量选择后端 -----
function createStore(kind) {
  const k = (kind || process.env.RANK_STORE || 'memory').toLowerCase();
  if (k === 'file') return createFileStore();
  if (k === 'upstash') return createUpstashStore();
  return createMemoryStore();
}

module.exports = { createStore, createMemoryStore, createFileStore, createUpstashStore, makeStore };
// ===== 文件边界：server/index.js（接续上一部分） =====
// server/index.js
// 零依赖 Node HTTP 服务（Vercel Serverless / 任意云函数 / 本地均可跑）。
//
// 兼容两种运行模式：
// ① Vercel Serverless：导出默认函数，由 Vercel 运行时调用
// ② 本地开发：node index.js 启动 HTTP 服务器（监听 PORT 或 3000）
//
// 接口：
// POST /api/score      body: { uid, name, score, ts, sig }
//                    -> 校验 HMAC 签名 + 时间戳 + 分数范围，记录最高分
// GET /api/rank?uid=xxx&limit=100&ts=xxx&sig=xxx

```

```
//          -> 校验签名后返回榜单视图
// POST /api/ladder/match  body: { uid, name, score, steps, boardSummary, ts, sig }
//          -> 异步天梯匹配（Phase 1）：写快照、匹配对手、比拼、写历史、返回结算
// GET /api/ladder/history?uid=&limit=&ts=&sig=
//          -> 校验签名后返回本人天梯战绩
//
// 环境变量：
// RANK_SECRET    —— 签名密钥（前后端一致）。未设置则「开发模式」跳过验签（仅本地测试用）
// STORAGE        —— 'memory'(默认) | 'file' | 'upstash'
// UPSTASH_REDIS_REST_URL / _TOKEN —— upstash 后端凭据
// RANK_MAX_SCORE —— 单局分数上限（防极端伪造，默认 10,000,000）
// SIGN_TTL       —— 签名有效期秒数（默认 300）

const http = require('http');
const { createStore } = require('./store.js');
const { verifyPayload } = require('./verify.js');
const { LadderService } = require('./ladder.js');
const { makeRoomService } = require('./room.js');

const RANK_SECRET = process.env.RANK_SECRET || '';
const SIGN_TTL = parseInt(process.env.SIGN_TTL || '300', 10);
const RANK_MAX_SCORE = parseInt(process.env.RANK_MAX_SCORE || '10000000', 10);
const store = createStore(process.env.STORAGE || process.env.RANK_STORE);

if (!RANK_SECRET) {
  console.warn('[rank-server] △ 未设置 RANK_SECRET：签名校验已关闭（仅限本地测试）。上线前务必设置随机密钥！');
}

// 统一 JSON 发送（含 CORS，沿用现有约定）
function sendJson(res, code, obj) {
  res.writeHead(code, {
    'Content-Type': 'application/json',
    'Access-Control-Allow-Origin': '*',
    'Access-Control-Allow-Methods': 'GET,POST,OPTIONS',
    'Access-Control-Allow-Headers': 'Content-Type',
  });
  res.end(JSON.stringify(obj));
}

// 天梯服务（复用全局 store 与 verify）
const ladder = new LadderService(store, { verifyPayload, maxScore: RANK_MAX_SCORE, send: sendJson });

// 房间对战服务（Phase 2，复用同一 store 实例与 RANK_SECRET）
const roomService = makeRoomService(store, RANK_SECRET);

// ----- 核心路由处理函数（Vercel + 本地共用） -----
function handleRequest(req, res) {
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Access-Control-Allow-Methods', 'GET,POST,OPTIONS');
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type');
  const send = (code, obj) => sendJson(res, code, obj);

  if (req.method === 'OPTIONS') { res.writeHead(204); res.end(); return; }

  // ----- 天梯路由（Phase 1） -----
  // POST /api/ladder/match
  if (req.method === 'POST' && req.url.startsWith('/api/ladder/match')) {
    return ladder.match(req, res);
  }
  // GET /api/ladder/history
  if (req.method === 'GET' && req.url.startsWith('/api/ladder/history')) {
    return ladder.getHistory(req, res);
  }
}
```



```
}

// ----- 旧接口（行为不变） -----
// POST /api/score
if (req.method === 'POST' && req.url.startsWith('/api/score')) {
  // Vercel Serverless 已预解析 body 为对象；本地模式需要手动拼接
  const rawBody = typeof req.body === 'string' ? req.body :
    (typeof req.body === 'object' ? JSON.stringify(req.body) : '');
  let body = rawBody;
  if (!body && !req.bodyUsed) {
    // 本地模式：手动读取流
    return new Promise((resolve) => {
      let buf = '';
      req.on('data', (c) => (buf += c));
      req.on('end', () => {
        handleScorePost(buf, send, resolve);
      });
    });
  }
  return handleScorePost(body, send, () => {});
}

// GET /api/rank
if (req.method === 'GET' && req.url.startsWith('/api/rank')) {
  return (async () => {
    const u = new URL(req.url, 'http://localhost');
    const uid = u.searchParams.get('uid') || '';
    const ts = u.searchParams.get('ts') || '';
    const sig = u.searchParams.get('sig') || '';
    const limit = parseInt(u.searchParams.get('limit') || '100', 10);
    if (!verifyPayload(uid + '|' + ts, sig)) {
      return send(403, { code: 403, message: 'invalid signature' });
    }
  })()
  try {
    const view = await store.getRankView(uid, limit);
    return send(200, { code: 0, data: view });
  } catch (e) {
    // upstash 不可达 / 出错：降级返回默认视图，绝不让前端卡死（前端会显示「排名暂不可用」）。
    // 注意：store.getRankView 仍按契约在 upstash 错误时抛错（store-upstash.test 已覆盖），
    // 此处仅在 HTTP 信封层兜底，保证 /api/rank 永远返回 200 + 结构化默认数据。
    return send(200, { code: 0, data: { top: [], selfRank: 0, selfName: '', selfScore: 0 } });
  }
}

// ----- 房间对战路由（Phase 2） -----
const roomMatch = req.url.match(/^\/api\/room\/([a-z]+)/);
if (roomMatch && ROOM_ROUTES[roomMatch[1]] && req.method === ROOM_ROUTES[roomMatch[1]].method) {
  return handleRoomRoute(roomMatch[1], req, res, send);
}

send(404, { code: 404, message: 'not found' });
}

// 提取 POST /api/score 处理逻辑（复用于 Vercel 和本地模式）
function handleScorePost(body, send, done) {
  try {
    const { uid, name, score, ts, sig } = JSON.parse(body || '{}');
    if (!uid) return send(400, { code: 400, message: 'uid required' });
    const sc = Math.floor(Number(score) || 0);
    if (!Number.isFinite(sc) || sc < 0 || sc > RANK_MAX_SCORE) {
      return send(400, { code: 400, message: 'score out of range' });
    }
  }
}
```

```

    }
    if (!verifyPayload(uid + '|' + sc + '|' + ts, ts, sig)) {
      return send(403, { code: 403, message: 'invalid signature' });
    }
    store.recordScore(uid, name, sc).then(() => {
      send(200, { code: 0, data: { ok: true } });
      done();
    }).catch((e) => {
      send(500, { code: 500, message: String(e && e.message || e) });
      done();
    });
  } catch (e) {
    send(400, { code: 400, message: String(e && e.message || e) });
    done();
  }
}

// ----- 房间对战路由辅助（Phase 2） -----
// 从请求体读取完整 JSON（兼容 Vercel 预解析 req.body 与本地流式）
function readBody(req) {
  return new Promise((resolve) => {
    if (req.body !== undefined) {
      const b = typeof req.body === 'string' ? req.body : (req.body ? JSON.stringify(req.body) : '');
      return resolve(b);
    }
    let buf = '';
    req.on('data', (c) => (buf += c));
    req.on('end', () => resolve(buf));
  });
}

// 房间路由表：method=HTTP 方法， svc=RoomService 方法名，
// sig=验签 canonical 拼接， args=提取传入 RoomService 的参数。
// canonical 严格对齐 design-lock § 4（create/join/state/leave → uid|ts；progress/result → uid|score|steps|ts）。
const ROOM_ROUTES = {
  create: { method: 'POST', svc: 'create', sig: (b) => String(b.uid) + '|' + b.ts, args: (b) => ({ uid: b.uid, name: b.name }) },
  join: { method: 'POST', svc: 'join', sig: (b) => String(b.uid) + '|' + b.ts, args: (b) => ({ code: b.code, uid: b.uid, name: b.name }) },
  progress: { method: 'POST', svc: 'progress', sig: (b) => String(b.uid) + '|' + b.score + '|' + b.steps + '|' + b.ts, args: (b) => ({ code: b.code, uid: b.uid, score: b.score, steps: b.steps, over: b.over }) },
  state: { method: 'GET', svc: 'getState', sig: (q) => String(q.uid) + '|' + q.ts, args: (q) => ({ code: q.code, uid: q.uid }) },
  result: { method: 'POST', svc: 'submitResult', sig: (b) => String(b.uid) + '|' + b.score + '|' + b.steps + '|' + b.ts, args: (b) => ({ code: b.code, uid: b.uid, score: b.score, steps: b.steps, won: b.won }) },
  leave: { method: 'POST', svc: 'leave', sig: (q) => String(q.uid) + '|' + q.ts, args: (q) => ({ code: q.code, uid: q.uid }) },
  // 再来一局（design-lock § 3 Q5）：终局后由先点击方触发 status→waiting；对手轮询到 waiting 后重连重开
  reset: { method: 'POST', svc: 'reset', sig: (b) => String(b.uid) + '|' + b.ts, args: (b) => ({ code: b.code, uid: b.uid }) },
};

async function handleRoomRoute(name, req, res, send) {
  const cfg = ROOM_ROUTES[name];
  let params;
  if (cfg.method === 'GET') {
    const u = new URL(req.url, 'http://localhost');
    params = {
      code: u.searchParams.get('code'),
      uid: u.searchParams.get('uid'),
      ts: u.searchParams.get('ts'),
      sig: u.searchParams.get('sig'),
    };
  } else {
    const raw = await readBody(req);
    try { params = JSON.parse(raw || '{}'); } catch (e) {
      return send(200, { code: 4, data: null, message: '请求体格式错误' });
    }
  }
}

```

```

// 验签 (secret 为空→开发模式跳过；对齐 verifyPayload)
if (!verifyPayload(cfg.sig(params), params.ts, params.sig)) {
  return send(200, { code: 2, data: null, message: '签名失效' });
}
try {
  const result = await roomService[cfg.svc](cfg.args(params));
  return send(200, result);
} catch (e) {
  return send(200, { code: 5, data: null, message: '服务端错误：' + (e && e.message || e) });
}
}

// ----- 导出：Vercel Serverless 模式 -----
// Vercel 运行时直接调用此函数（不需要 http.createServer）
module.exports = handleRequest;
// 同时支持 module.exports.default（部分 Vercel 版本偏好）
module.exports.default = handleRequest;

// ----- 本地开发模式：启动 HTTP 服务器 -----
if (require.main === module) {
  const PORT = process.env.PORT || 3000;
  const server = http.createServer(handleRequest);
  server.listen(PORT, () => console.log('[rank-server] listening on http://localhost:' + PORT + ' (store=' + (store._backend) + ')'));
}
// ===== 文件边界：server/room.js（接续上一部分） =====
// server/room.js
// Phase 2 实时房间对战 —— RoomService（房间状态机 + 终局权威判定 + 再来一局）。
//
// 纯逻辑，零新增 npm 依赖；复用全局 store（memory / file / upstash 三后端）与 verifyPayload。
//
// 信封：{ code, data, message }
// code: 0 成功 / 1 参数错 / 2 签名失效(由 index.js 判定) / 3 未找到 / 4 房间已满或频率 / 5 服务端错
// 存储键：room:<code>（整体 JSON，EXPIRE 600s，读后 roomTouch 续期）
//
// 服务端只负责「房间状态」与「终局权威判定」：
// - seed 仅由服务端 create 生成并下发；RNG 在客户端跑（src/logic.js makeRng），服务端绝不跑 PRNG。
// - 胜负规则（design-lock § 3 Q1/Q2）：先到 2048 者胜 → 否则 180s 到点比分高者胜 → 平分平局。

'use strict';

// ----- 锁定的服务端常量（design-lock § 1.1） -----
const ROOM_PLAYERS = 2; // 房间固定 2 人
const ROOM_TTL_SECONDS = 600; // Redis key EXPIRE（重连窗口）
const ROOM_CODE_LEN = 6; // 房码长度（固定 6 位）
const ROOM_CODE_CHARS = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789'; // 仅大写字母 + 数字
const ROOM_CODE_REGEX = /^[A-Z0-9]{6}$/; // 加入时校验房码格式
const SYNC_BUFFER_MS = 3000; // 第 2 人加入: startAt = Date.now() + 3000
const MATCH_DURATION_MS = 180000; // 180s 到点判定边界 = startAt + 180000
const STALE_ABANDON_MS = 600000; // 对手 lastSeen 超过此值(>TTL) → abandoned
const PROGRESS_THROTTLE_MS = 500; // progress 节流下限（或每步取先到者，服务端不强制）
const SEED_MAX = 0x7FFFFFFF; // makeSeed() 取值范围 [0, SEED_MAX]

// ----- 终局权威判定（幂等，design-lock § 4.7） -----
// 返回 1(P1胜) / 2(P2胜) / 3(平局)。P1 = uids[0]（先加入者），P2 = uids[1]。
function computeMatch(room) {
  const uids = Object.keys(room.players || {});
  if (uids.length < 2) return 3; // 异常兜底（理论上不会进入）
  // 1) 先到 2048：任一方 won=true → 该方胜
  for (const u of uids) {
    if (room.results[u] && room.results[u].won) {
      return u === uids[0] ? 1 : 2;
    }
  }
}

```

```
}
// 2) 到点：比最新分数（来自 players 的最新一手快照）
const s0 = (room.players[uids[0]] && room.players[uids[0]].score) || 0;
const s1 = (room.players[uids[1]] && room.players[uids[1]].score) || 0;
if (s0 > s1) return 1;
if (s1 > s0) return 2;
return 3; // 平分
}

// 组装 getState 的 data 载荷（design-lock § 4.4）
function buildStateData(room, uid, opp, matchResult) {
  const myP = room.players[String(uid)] || {};
  const oppP = opp ? room.players[opp] : {};
  return {
    status: room.status,
    startAt: room.startAt,
    seed: room.seed,
    matchResult: matchResult || 0,
    myScore: myP.score || 0,
    oppScore: oppP.score || 0,
    opponent: opp ? {
      score: oppP.score || 0,
      steps: oppP.steps || 0,
      over: !oppP.over,
      updatedAt: oppP.updatedAt || 0,
    } : null,
  };
}

// ----- RoomService 工厂 -----
// @param {object} store 由 createStore 创建的全局存储实例（须含房间方法）
// @param {string} secret 签名密钥（RANK_SECRET）；验签在 index.js 完成，此处仅保留引用
function makeRoomService(store, secret) {
  // secret 预留：与 verifyPayload 共用 RANK_SECRET，验签由 index.js 负责

  // 生成 6 位房码：循环至 roomGet 为 null（ $34^6 \approx 1.5e9$  组合，memory 下极难碰撞）
  async function makeCode() {
    for (let i = 0; i < 50; i++) {
      let code = "";
      for (let j = 0; j < ROOM_CODE_LEN; j++) {
        code += ROOM_CODE_CHARS[Math.floor(Math.random() * ROOM_CODE_CHARS.length)];
      }
      const exists = await store.roomGet(code);
      if (!exists) return code;
    }
    // 兜底：极偶然碰撞时追加一位，避免无限循环
    let code = "";
    for (let j = 0; j < ROOM_CODE_LEN; j++) {
      code += ROOM_CODE_CHARS[Math.floor(Math.random() * ROOM_CODE_CHARS.length)];
    }
    code += ROOM_CODE_CHARS[Math.floor(Math.random() * ROOM_CODE_CHARS.length)];
    return code;
  }

  // 生成整数 seed  $\in [0, \text{SEED\_MAX}]$ 
  function makeSeed() {
    return Math.floor(Math.random() * (SEED_MAX + 1));
  }

  return {
    makeCode,
    makeSeed,
  };
}
```

```
computeMatch,

// POST /api/room/create —— 建房（design-lock § 4.1）
async create({ uid, name }) {
  if (!uid) return { code: 1, data: null, message: '缺少 uid' };
  const code = await makeCode();
  const seed = makeSeed();
  const now = Date.now();
  const room = {
    code,
    seed,
    status: 'waiting',
    startAt: null,
    orphan: false,
    players: {
      [String(uid)]: {
        uid: String(uid),
        name: String(name || '玩家').slice(0, 16),
        ready: true,
        score: 0,
        steps: 0,
        over: false,
        updatedAt: now,
        left: false,
      },
    },
    results: {},
    matchResult: 0,
  };
  await store.roomSet(code, room);
  return {
    code: 0,
    data: { code, seed, status: 'waiting', startAt: null, ttl: ROOM_TTL_SECONDS },
    message: 'ok',
  };
},

// POST /api/room/join —— 加入 / 重连（design-lock § 4.2）
async join({ code, uid, name }) {
  if (!code || !uid) return { code: 1, data: null, message: '缺少 code 或 uid' };
  if (!ROOM_CODE_REGEX.test(String(code))) return { code: 1, data: null, message: '房码格式非法' };
  const room = await store.roomGet(code);
  if (!room) return { code: 3, data: null, message: '房间不存在或已过期' };
  const now = Date.now();
  const uidStr = String(uid);
  // 重连：已存在则刷新 updatedAt、清 left（不重复加入，AC-7）
  if (room.players[uidStr]) {
    room.players[uidStr].updatedAt = now;
    room.players[uidStr].left = false;
  } else {
    // 满员（且非终局后 reset 重开场景）
    if (Object.keys(room.players).length >= ROOM_PLAYERS) {
      return { code: 4, data: null, message: '房间已满' };
    }
    room.players[uidStr] = {
      uid: uidStr,
      name: String(name || '玩家').slice(0, 16),
      ready: true,
      score: 0,
      steps: 0,
      over: false,
      updatedAt: now,
    };
  }
}
```

```

    left: false,
  };
}
// 凑满 2 人 → 首局 playing；或终局后 reset 回到 waiting 且双方仍在 → 重开 playing
// （design-lock § 3 Q5：再来一局后任一方重连即重开，写新 startAt）
if (room.status === 'waiting' && Object.keys(room.players).length >= ROOM_PLAYERS) {
  room.status = 'playing';
  room.startAt = now + SYNC_BUFFER_MS;
  room.orphan = false;
}
await store.roomSet(code, room);
return {
  code: 0,
  data: { seed: room.seed, players: room.players, status: room.status, startAt: room.startAt },
  message: 'ok',
};
},

// POST /api/room/progress —— 落子进度上报（design-lock § 4.3）
async progress({ code, uid, score, steps, over }) {
  if (!code || !uid) return { code: 1, data: null, message: '缺少 code 或 uid' };
  const room = await store.roomGet(code);
  if (!room) return { code: 3, data: null, message: '房间不存在或已过期' };
  if (room.status !== 'playing') return { code: 3, data: null, message: '当前不在对局中' };
  await store.roomProgress(code, uid, {
    score: Math.floor(Number(score) || 0),
    steps: Math.floor(Number(steps) || 0),
    over: !!over,
  });
  await store.roomTouch(code); // 续期 600s
  return { code: 0, data: { ok: true }, message: 'ok' };
},

// GET /api/room/state —— 轮询房间状态（含终局判定，design-lock § 4.4）
async getState({ code, uid }) {
  if (!code || !uid) return { code: 1, data: null, message: '缺少 code 或 uid' };
  const room = await store.roomGet(code);
  if (!room) return { code: 3, data: null, message: '房间不存在或已过期' };
  await store.roomTouch(code); // 续期 600s
  const now = Date.now();
  const uids = Object.keys(room.players);
  const opp = uids.length === 2 ? uids.find((u) => u !== String(uid)) : null;
  // 仅当仍为 playing 才做终局判定（left → stale → results 齐 → timeout），幂等
  if (room.status === 'playing') {
    if (opp && room.players[opp] && room.players[opp].left === true) {
      room.matchResult = 4; // 对手主动离开（我方胜）
      room.status = 'opponent_left';
    } else if (opp && room.players[opp] && (now - (room.players[opp].updatedAt || 0)) > STALE_ABANDON_MS) {
      room.matchResult = 5; // 对手断线超 TTL（我方胜）
      room.status = 'abandoned';
    } else if (uids.length === 2 && room.results[uids[0]] && room.results[uids[1]]) {
      room.matchResult = computeMatch(room); // 双方结果齐
      room.status = 'finished';
    } else if (now >= (room.startAt || 0) + MATCH_DURATION_MS) {
      room.matchResult = computeMatch(room); // 180s 到点
      room.status = 'finished';
    }
  }
  if (room.matchResult) await store.roomSet(code, room);
}
return { code: 0, data: buildStateData(room, String(uid), opp, room.matchResult), message: 'ok' };
},

```

```
// POST /api/room/result — 提交终局结果 (design-lock § 4.5)
async submitResult({ code, uid, score, steps, won }) {
  if (!code || !uid) return { code: 1, data: null, message: '缺少 code 或 uid' };
  const room = await store.roomGet(code);
  if (!room) return { code: 3, data: null, message: '房间不存在或已过期' };
  const now = Date.now();
  room.results[String(uid)] = {
    uid: String(uid),
    score: Math.floor(Number(score) || 0),
    steps: Math.floor(Number(steps) || 0),
    ts: Math.floor(now / 1000),
    won: !!won,
  };
  const uids = Object.keys(room.players);
  // 仅当仍为 playing 且双方结果齐才写终局（幂等，避免覆盖 opponent_left/abandoned）
  if (room.status === 'playing' && uids.length === 2 && room.results[uids[0]] && room.results[uids[1]]) {
    room.matchResult = computeMatch(room);
    room.status = 'finished';
  }
  await store.roomSet(code, room);
  return { code: 0, data: { ok: true }, message: 'ok' };
},

// POST /api/room/leave — 离开房间 (design-lock § 4.6 / § 3 Q6)
async leave({ code, uid }) {
  if (!code || !uid) return { code: 1, data: null, message: '缺少 code 或 uid' };
  const room = await store.roomGet(code);
  if (!room) return { code: 3, data: null, message: '房间不存在或已过期' };
  if (room.status === 'waiting') {
    // waiting 期退出：移除槽位，房间保留（清空则标记 orphan）
    delete room.players[String(uid)];
    if (Object.keys(room.players).length === 0) room.orphan = true;
    await store.roomSet(code, room);
  } else if (room.status === 'playing') {
    // playing 期退出：标记 left，写 matchResult=4、status=opponent_left
    if (!room.players[String(uid)]) {
      room.players[String(uid)] = { uid: String(uid), name: '玩家', score: 0, steps: 0, over: false, updatedAt: Date.now() };
    }
    room.players[String(uid)].left = true;
    room.matchResult = 4;
    room.status = 'opponent_left';
    await store.roomSet(code, room);
  }
  // 其余终局态（finished/opponent_left/abandoned）：不改终局，仅返回 ok
  return { code: 0, data: { ok: true }, message: 'ok' };
},

// POST /api/room/reset — 再来一局 (design-lock § 3 Q5)
async reset({ code, uid }) {
  if (!code) return { code: 1, data: null, message: '缺少 code' };
  const room = await store.roomGet(code);
  if (!room) return { code: 3, data: null, message: '房间不存在或已过期' };
  // 仅终局后允许（matchResult ∈ {1,2,3,4,5}）
  if (![1, 2, 3, 4, 5].includes(room.matchResult)) {
    return { code: 4, data: null, message: '对局尚未结束，无法重开' };
  }
  const now = Date.now();
  const newSeed = makeSeed();
  // 清空双方进度（保留 name），新 seed、回 waiting
  for (const u of Object.keys(room.players)) {
    room.players[u] = {
      uid: u,
```

```

    name: room.players[u].name || '玩家',
    ready: true,
    score: 0,
    steps: 0,
    over: false,
    updatedAt: now,
    left: false,
  };
}
room.seed = newSeed;
room.results = {};
room.matchResult = 0;
room.status = 'waiting';
room.startAt = null;
room.orphan = false;
await store.roomSet(code, room); // 续 600s
return { code: 0, data: { seed: newSeed, status: 'waiting' }, message: 'ok' };
},
};
}

module.exports = { makeRoomService, computeMatch, ROOM_CODE_REGEX };
// ===== 文件边界：server/ladder.js（接续上一部分） =====
// server/ladder.js
// 天梯（异步匹配）Phase 1 服务端：LadderService
// 路由入口在 server/index.js：
// POST /api/ladder/match —— 写本人快照、按分数带宽匹配对手（无则合成降级）、比拼、写历史、返回结算
// GET /api/ladder/history —— 验签后返回本人战绩（按 ts 倒序）
//
// 信封：{ code, data, message }
// code: 0 成功 / 1 参数错 / 2 签名失效 / 3 未找到 / 5 服务端错
// 复用全局 store 实例与 verifyPayload（HMAC-SHA256，密钥 RANK_SECRET）。
//
// 设计要点：
// - 旧接口 /api/score、/api/rank 仍使用 { ok, error } 信封，本服务不改动它们。
// - 匹配带宽是服务端算的：band = max(50, round(score * 0.15))，即 ±15% 且最小 ±50 分。
// - 合成对手（synthetic）：池空时生成“神秘高手”，name 随机、分数贴近本人，保证有对手可玩。

const CODE = { OK: 0, PARAM: 1, SIGN: 2, NOT_FOUND: 3, SERVER: 5 };

// 从 http.IncomingMessage 读取完整请求体（兼容真实流式与测试用同步 fake req）
function readBody(req) {
  return new Promise((resolve) => {
    let body = '';
    req.on('data', (c) => (body += c));
    req.on('end', () => resolve(body));
  });
}

class LadderService {
  /**
   * @param {object} store 由 createStore 创建的全局存储实例（须含天梯方法）
   * @param {object} opts { verifyPayload, maxScore, send }
   * - verifyPayload(payload, ts, sig) -> boolean
   * - maxScore 单局分数上限（默认 10,000,000）
   * - send(res, code, obj) 可注入带 CORS 的发送函数（index.js 注入）；缺省自带 CORS。
   */
  constructor(store, opts = {}) {
    this.store = store;
    this.verifyPayload = opts.verifyPayload || (() => true);
    this.maxScore = opts.maxScore || 10000000;
    this._send = opts.send || null;
  }

```



```
}

// 统一 JSON 发送（含 CORS，沿用现有约定）
send(res, code, obj) {
  if (this._send) return this._send(res, code, obj);
  res.writeHead(code, {
    'Content-Type': 'application/json',
    'Access-Control-Allow-Origin': '*',
    'Access-Control-Allow-Methods': 'GET,POST,OPTIONS',
    'Access-Control-Allow-Headers': 'Content-Type',
  });
  res.end(JSON.stringify(obj));
}

// 生成合成对手（池空降级）。分数贴近本人，制造势均力敌的体验。
makeSynthetic(score) {
  const sc = Math.floor(Number(score) || 0);
  const band = Math.max(50, Math.round(sc * 0.15));
  const delta = Math.floor((Math.random() * 2 - 1) * band);
  const oppScore = Math.max(0, sc + delta);
  const steps = Math.max(1, Math.round(sc / 4 + Math.random() * 20));
  const names = ['神秘高手', '隐世宗师', '无名强者', '天梯幻影', '合成大师'];
  const name = names[Math.floor(Math.random() * names.length)];
  return { uid: 'synthetic', name, score: oppScore, steps, boardSummary: null, synthetic: true };
}

// POST /api/ladder/match
async match(req, res) {
  let payload;
  try {
    payload = JSON.parse((await readBody(req)) || '{}');
  } catch (e) {
    return this.send(res, 400, { code: CODE.PARAM, data: null, message: 'invalid json' });
  }

  const { uid, name, score, steps, boardSummary, ts, sig } = payload;

  // 1) 参数校验
  if (!uid) return this.send(res, 400, { code: CODE.PARAM, data: null, message: 'uid required' });
  // 必填字段 score 缺失：按架构约定（docs/system_design.md § 7）返回 code 1（PARAM）。
  // 用 == null 而非 falsy，以保留 score=0 这一合法分数值。
  if (score == null) return this.send(res, 400, { code: CODE.PARAM, data: null, message: 'score required' });
  // score 必须是有限数值（必填数值）。字符串（含 '/abc/'123abc'）一律视为缺失，
  // 禁止被 Number() 静默 coerce 成 0 后当合法 score=0 落库（详见 test/ladder-nan.test.js）。
  // 注意：Number("") === 0（非 NaN），故不能仅靠 !Number.isFinite 捕获空串，需显式按类型拒绝。
  if (typeof score !== 'number' || !Number.isFinite(score)) {
    return this.send(res, 400, { code: CODE.PARAM, data: null, message: 'score required' });
  }
  const sc = Math.floor(score);
  // steps 同理：必须为有限数值，非数字字符串一律拒绝（与 score 保持一致）。
  if (typeof steps !== 'number' || !Number.isFinite(steps) || steps < 0) {
    return this.send(res, 400, { code: CODE.PARAM, data: null, message: 'steps invalid' });
  }
  const st = Math.floor(steps);
  if (sc < 0 || sc > this.maxScore) {
    return this.send(res, 400, { code: CODE.PARAM, data: null, message: 'score out of range' });
  }
  if (ts == null) return this.send(res, 400, { code: CODE.PARAM, data: null, message: 'ts required' });

  // 2) 验签：canonical = uid|score|steps|ts
  const canonical = String(uid) + '|' + sc + '|' + st + '|' + ts;
  if (!this.verifyPayload(canonical, ts, sig)) {
```

```
    return this.send(res, 403, { code: CODE.SIGN, data: null, message: 'invalid signature' });
  }

  try {
    // 3) 写本人快照（供他人匹配）
    await this.store.saveSnapshot({
      uid: String(uid),
      name: String(name || '玩家').slice(0, 16),
      score: sc,
      steps: st,
      boardSummary: boardSummary == null ? null : boardSummary,
      ts: Number(ts),
      synthetic: false,
    });

    // 4) 匹配对手（band 默认 ±15% 且最小 ±50 分）
    const band = Math.max(50, Math.round(sc * 0.15));
    let opponent = await this.store.matchSnapshot(sc, uid, band);
    let synthetic = false;
    if (!opponent) {
      opponent = this.makeSynthetic(sc);
      synthetic = true;
    }

    // 5) 比拼：用已签名的本局 score 与对手 score 比较
    const oppScore = Math.floor(Number(opponent.score) || 0);
    const diff = sc - oppScore;
    let result = 'draw';
    if (diff > 0) result = 'win';
    else if (diff < 0) result = 'loss';

    const matchId = 'm' + Date.now().toString(36) + Math.floor(Math.random() * 1e6).toString(36);

    // 6) 写历史（本人）
    const rec = {
      matchId,
      myScore: sc,
      oppName: String(opponent.name || '对手'),
      oppScore,
      oppUid: String(opponent.uid || 'synthetic'),
      opponentSynthetic: !!synthetic,
      result,
      diff,
      ts: Number(ts),
    };
    await this.store.pushHistory(uid, rec);

    // 7) 返回结算卡数据
    const data = {
      matchId,
      myScore: sc,
      opponent: {
        name: String(opponent.name || '对手'),
        score: oppScore,
        steps: Math.floor(Number(opponent.steps) || 0),
        boardSummary: opponent.boardSummary == null ? null : opponent.boardSummary,
        synthetic: !!synthetic,
      },
      result,
      diff,
      synthetic,
    };
  }
```

```

    return this.send(res, 200, { code: CODE.OK, data, message: 'ok' });
  } catch (e) {
    return this.send(res, 500, { code: CODE.SERVER, data: null, message: String(e) + e.message || e });
  }
}

// GET /api/ladder/history?uid=&limit=&ts=&sig=
async getHistory(req, res) {
  try {
    const u = new URL(req.url, 'http://localhost');
    const uid = u.searchParams.get('uid') || '';
    const ts = u.searchParams.get('ts') || '';
    const sig = u.searchParams.get('sig') || '';
    const limit = parseInt(u.searchParams.get('limit') || '20', 10);

    if (!uid) return this.send(res, 400, { code: CODE.PARAM, data: null, message: 'uid required' });
    // 验签：canonical = uid|ts （与排行榜一致）
    if (!this.verifyPayload(String(uid) + '|' + ts, sig)) {
      return this.send(res, 403, { code: CODE.SIGN, data: null, message: 'invalid signature' });
    }

    const full = await this.store.getHistory(uid); // 取全部（最多 50，由 store LTRIM 保证）
    const total = full.length;
    const list = full.slice(0, Math.max(1, Math.min(50, limit || 20)));
    return this.send(res, 200, {
      code: CODE.OK,
      data: { list, total },
      message: 'ok',
    });
  } catch (e) {
    return this.send(res, 500, { code: CODE.SERVER, data: null, message: String(e) + e.message || e });
  }
}

module.exports = { LadderService, CODE };
// ===== 文件边界：server/worker.js（接续上一部分） =====
// server/worker.js —— Cloudflare Workers 入口（fetch 风格）
// 部署：wrangler publish（在 wrangler.toml 里把 main 指向本文件）
const { recordScore, getRankView } = require('./store.js');

function json(body, code) {
  return new Response(JSON.stringify(body), {
    status: code || 200,
    headers: { 'Content-Type': 'application/json', 'Access-Control-Allow-Origin': '*' },
  });
}

async function handle(req) {
  const url = new URL(req.url);
  if (req.method === 'OPTIONS') {
    return new Response(null, { status: 204, headers: { 'Access-Control-Allow-Origin': '*' } });
  }
  if (req.method === 'POST' && url.pathname === '/api/score') {
    const { uid, name, score } = await req.json().catch(() => ({}));
    if (!uid) return json({ ok: false, error: 'uid required' }, 400);
    recordScore(uid, name, score);
    return json({ ok: true });
  }
  if (req.method === 'GET' && url.pathname === '/api/rank') {
    const uid = url.searchParams.get('uid') || '';
    const limit = parseInt(url.searchParams.get('limit') || '100', 10);
  }
}

```

```
    return json(getRankView(uid, limit));
  }
  return json({ ok: false, error: 'not found' }, 404);
}

module.exports = { fetch: handle };
// ===== 文件边界：server/verify.js（接续上一部分） =====
// server/verify.js
// 从 server/index.js 原样抽取的「请求签名校验」逻辑，供 /api/score、/api/rank、
// /api/ladder/* 等多个接口复用。行为与原内联实现逐字一致（secret 为空→开发跳过、
// 时间窗校验、HMAC-SHA256 + timingSafeEqual），不得改动。
const crypto = require('crypto');

const RANK_SECRET = process.env.RANK_SECRET || '';
const SIGN_TTL = parseInt(process.env.SIGN_TTL || '300', 10);

// 验签：secret 为空（开发模式）直接通过；否则校验签名 + 时间窗
function verifyPayload(payload, ts, sig) {
  if (!RANK_SECRET) return true;
  if (!sig || ts == null) return false;
  const t = parseInt(ts, 10);
  if (!Number.isFinite(t)) return false;
  const now = Math.floor(Date.now() / 1000);
  if (Math.abs(now - t) > SIGN_TTL) return false; // 过期 / 重放
  const expected = crypto.createHmac('sha256', RANK_SECRET).update(payload).digest('hex');
  const a = Buffer.from(expected);
  const b = Buffer.from(String(sig));
  if (a.length !== b.length) return false;
  try { return crypto.timingSafeEqual(a, b); } catch (e) { return false; }
}

module.exports = { verifyPayload, RANK_SECRET, SIGN_TTL };
// ===== 文件边界：game.js（接续上一部分） =====
  return;
}
}
// 排行榜按钮：命中即打开榜单
{
  const rb = rankBtnRect();
  if (screen === 'play' && t.clientX >= rb.x && t.clientX <= rb.x + rb.w &&
    t.clientY >= rb.y && t.clientY <= rb.y + rb.h) {
    openRank();
    return;
  }
}
// 天梯入口按钮（左上角，紧挨排行榜按钮右侧，避开系统胶囊）：命中即打开战绩面板
{
  const lb = Ladder.ladderEntryBtnRect();
  if (screen === 'play' && hit(lb, t)) {
    openLadderHistory();
    return;
  }
}
// 房间入口按钮（左上角，紧挨天梯按钮右侧，避开系统胶囊）：命中即进入房间大厅
// 修复 Bug2：此前该按钮仅绘制、无点击处理，导致「房间点不进去」。
{
  const rb2 = roomEntryBtnRect();
  if (screen === 'play' && t.clientX >= rb2.x && t.clientX <= rb2.x + rb2.w &&
    t.clientY >= rb2.y && t.clientY <= rb2.y + rb2.h) {
    openRoom();
    return;
  }
}
```

```

    }
    if (state.over) {
        watchRewardedThenRestart();
        return;
    }
    const dx = t.clientX - sx;
    const dy = t.clientY - sy;
    if (Math.abs(dx) < 24 && Math.abs(dy) < 24) return; // 视作点击，忽略
    let dir;
    if (Math.abs(dx) > Math.abs(dy)) dir = dx > 0 ? 'right' : 'left';
    else dir = dy > 0 ? 'down' : 'up';
    const res = Logic.move(state, dir);
    if (res.moved) {
        state = res.state;
        moveSteps += 1;
        applyMoveFx(res);
        if (state.over) { triggerGameOver(); }
    } else if (res.over) {
        // 棋盘已无路可走（满格且无相邻可合并），本次滑动未改变棋盘 → 判负
        state = Object.assign({}, state, { over: true });
        triggerGameOver();
    }
    });

// 排行榜界面：拖动滚动列表（手指上滑 → 列表上滚 → rankScroll 增大）
tt.onTouchMove((e) => {
    if (screen !== 'rank' && screen !== 'ladderHistory') return;
    const ty = e.touches[0].clientY;
    if (screen === 'rank') {
        rankScroll += (lastMoveY - ty);
        lastMoveY = ty;
    } else if (screen === 'ladderHistory') {
        ladderScroll += (lastMoveYL - ty);
        lastMoveYL = ty;
    }
});

// 系统返回（Android 返回键 / iOS 返回手势）：排行榜界面消费返回事件，仅关闭榜单不退出小游戏。
// 抖音基础库存在新旧两种 API 形态，需兼容：
// 新版：enableBackPressed() 开启监听（无参）+ onBackPressed(cb) 注册回调
// 旧版：enableBackPressed(cb) 直接把回调当作参数
// 回调返回 true = 消费事件（拦截，不退出）；false = 放行系统默认（退出小游戏）。
const onBack = () => {
    if (screen === 'rank' || screen === 'ladder' || screen === 'ladderHistory') {
        screen = 'play';
        return true; // 消费返回事件，仅关闭弹层（排行榜 / 天梯结算 / 天梯战绩）
    }
    if (screen === 'room') {
        Room.exit(); // 退出房间（playing/waiting 期礼貌 POST leave），返回主界面
        return true; // 消费返回事件，仅关闭房间
    }
    return false; // 游戏主界面走系统默认（退出小游戏）
};

if (typeof tt.onBackPressed === 'function') {
    try {
        tt.onBackPressed(onBack);
        if (typeof tt.enableBackPressed === 'function') {
            try { tt.enableBackPressed(); } catch (e) { /* 忽略 */ }
        }
    } catch (e) { /* 忽略，仍可点 × 返回 */ }
} else if (typeof tt.enableBackPressed === 'function') {
    // 兼容旧版：enableBackPressed(cb)

```

```
    try { tt.enableBackPressed(onBack); } catch (e) { /* 忽略, 仍可点 × 返回 */ }
  }
}

// ----- 视觉工具 -----
function roundRect(x, y, w, h, r) {
  ctx.beginPath();
  ctx.moveTo(x + r, y);
  ctx.arcTo(x + w, y, x + w, y + h, r);
  ctx.arcTo(x + w, y + h, x, y + h, r);
  ctx.arcTo(x, y + h, x, y, r);
  ctx.arcTo(x, y, x + w, y, r);
  ctx.closePath();
}

// 把十六进制颜色提亮 amt(0~1), 返回 rgb 字符串, 供火花/光晕使用
function lighten(hex, amt) {
  const n = parseInt(hex.slice(1), 16);
  let r = (n >> 16) & 255, g = (n >> 8) & 255, b = n & 255;
  r = Math.min(255, Math.round(r + (255 - r) * amt));
  g = Math.min(255, Math.round(g + (255 - g) * amt));
  b = Math.min(255, Math.round(b + (255 - b) * amt));
  return 'rgb(' + r + ',' + g + ',' + b + ')';
}

function cellTop(r, c) {
  return { x: boardX + PAD + c * (cell + PAD), y: boardY + PAD + r * (cell + PAD) };
}

function cellCenter(r, c) {
  const t = cellTop(r, c);
  return { x: t.x + cell / 2, y: t.y + cell / 2 };
}

// 震动开关按钮的屏幕矩形 (棋盘正下方居中)
function vibrateToggleRect() {
  const w = 104, h = 30;
  return { x: W / 2 - w / 2, y: boardY + boardSize + 28, w, h };
}

// 点中判定: 触摸点 t 是否落在矩形 r 内 (天梯/排行榜按钮复用)
function hit(r, t) {
  return t.clientX >= r.x && t.clientX <= r.x + r.w &&
    t.clientY >= r.y && t.clientY <= r.y + r.h;
}

function spawnParticles(r, c, val) {
  const ct = cellCenter(r, c);
  const base = COLORS[val] || '#3c3a32';
  const spark = lighten(base, 0.55);
  const n = 12;
  for (let i = 0; i < n; i++) {
    const ang = (Math.PI * 2 * i) / n + Math.random() * 0.5;
    const sp = cell * 0.003 + Math.random() * cell * 0.004;
    particles.push({
      x: ct.x, y: ct.y,
      vx: Math.cos(ang) * sp, vy: Math.sin(ang) * sp,
      life: 0, maxLife: 360 + Math.random() * 180,
      color: spark, size: cell * 0.05 + Math.random() * cell * 0.05,
    });
  }
}
```

```

function applyMoveFx(res) {
  if (!res || !res.merged || !res.merged.length) return;
  const cells = res.merged.map(([, c]) => ({ r, c, val: state.grid[r][c] }));
  mergeFx.push({ cells, t0: Date.now() });
  for (const cc of cells) spawnParticles(cc.r, cc.c, cc.val);
  if (res.gained) {
    scorePops.push({ x: boardX + boardSize / 2, y: boardY - 14, val: res.gained, t0: Date.now() });
  }
  // 合并时轻微震动（受震动开关控制），强化“撞击”手感
  try { if (vibrateOn && tt.vibrateShort) tt.vibrateShort({ type: 'light' }); } catch (e) { /* noop */ }
}

function updateParticles(dt) {
  for (let i = particles.length - 1; i >= 0; i--) {
    const p = particles[i];
    p.life += dt;
    if (p.life >= p.maxLife) { particles.splice(i, 1); continue; }
    p.vx -= p.vx * 0.012 * dt;
    p.vy -= p.vy * 0.012 * dt;
    p.vy += cell * 0.0006 * dt; // 轻微重力，粒子下坠更有质感
    p.x += p.vx * dt;
    p.y += p.vy * dt;
  }
}

function drawGlow(cx, cy, val, k) {
  const rad = cell * (0.9 + k * 0.7);
  const g = ctx.createRadialGradient(cx, cy, cell * 0.2, cx, cy, rad);
  g.addColorStop(0, 'rgba(255,236,150,' + (0.55 * (1 - k)).toFixed(3) + ')');
  g.addColorStop(0.5, 'rgba(246,178,89,' + (0.30 * (1 - k)).toFixed(3) + ')');
  g.addColorStop(1, 'rgba(246,178,89,0)');
  ctx.fillStyle = g;
  ctx.beginPath();
  ctx.arc(cx, cy, rad, 0, Math.PI * 2);
  ctx.fill();
}

function drawRing(cx, cy, k) {
  ctx.save();
  ctx.globalAlpha = (1 - k) * 0.7;
  ctx.strokeStyle = 'rgba(255,239,160,0.95)';
  ctx.lineWidth = cell * 0.10 * (1 - k) + 1;
  ctx.beginPath();
  ctx.arc(cx, cy, cell * (0.5 + k * 0.7), 0, Math.PI * 2);
  ctx.stroke();
  ctx.restore();
}

function drawCell(x, y, val, scale, flash) {
  const s = scale || 1;
  const w = cell * s;
  const off = (w - cell) / 2;
  const px = x - off, py = y - off;
  ctx.save();
  ctx.shadowColor = 'rgba(0,0,0,0.18)';
  ctx.shadowBlur = cell * 0.06;
  ctx.shadowOffsetY = cell * 0.03;
  ctx.fillStyle = val ? (COLORS[val] || '#3c3a32') : '#cdc1b4';
  roundRect(px, py, w, w, cell * 0.12);
  ctx.fill();
  ctx.restore();
  if (flash && flash > 0) {

```

```
ctx.save();
ctx.globalAlpha = flash;
ctx.fillStyle = '#ffffff';
roundRect(px, py, w, w, cell * 0.12);
ctx.fill();
ctx.restore();
}
if (val) {
  ctx.fillStyle = val <= 4 ? '#776e65' : '#f9f6f2';
  ctx.font = 'bold ' + Math.floor(cell * 0.38 * s) + 'px sans-serif';
  ctx.textAlign = 'center';
  ctx.textBaseline = 'middle';
  ctx.fillText(String(val), px + w / 2, py + w / 2);
}
}

function draw() {
  const now = Date.now();
  mergeFx = mergeFx.filter((fx) => now - fx.t0 < POP_MS);
  scorePops = scorePops.filter((s) => now - s.t0 < SCORE_MS);

  ctx.clearRect(0, 0, W, H);
  ctx.fillStyle = '#faf8ef';
  ctx.fillRect(0, 0, W, H);

  // 顶部标题与分数
  ctx.fillStyle = '#776e65';
  ctx.textAlign = 'left';
  ctx.textBaseline = 'alphabetic';
  ctx.font = 'bold 22px sans-serif';
  ctx.fillText('合成能量', boardX, boardY - 36);
  ctx.font = 'bold 18px sans-serif';
  ctx.textAlign = 'right';
  ctx.fillText('分数' + state.score, boardX + boardSize, boardY - 36);

  // 棋盘底板
  ctx.fillStyle = '#bbada0';
  roundRect(boardX, boardY, boardSize, boardSize, cell * 0.12);
  ctx.fill();

  // 合并特效映射：key = r*SIZE+c
  const mmap = {};
  for (const fx of mergeFx) {
    const k = (now - fx.t0) / POP_MS;
    if (k >= 1) continue;
    for (const cc of fx.cells) {
      mmap[cc.r * SIZE + cc.c] = {
        scale: 1 + 0.22 * Math.sin(k * Math.PI), // 弹性放大：1 → 1.22 → 1
        flash: Math.max(0, 1 - k) * 0.5,
        val: cc.val,
        k,
      };
    }
  }

  // 第一遍：光晕（在方块之下）
  for (let r = 0; r < SIZE; r++) {
    for (let c = 0; c < SIZE; c++) {
      const m = mmap[r * SIZE + c];
      if (!m) continue;
      const ct = cellCenter(r, c);
      drawGlow(ct.x, ct.y, m.val, m.k);
    }
  }
}
```



```

    }
  }

  // 第二遍：方块本体
  for (let r = 0; r < SIZE; r++) {
    for (let c = 0; c < SIZE; c++) {
      const t = cellTop(r, c);
      const m = mmap[r * SIZE + c];
      drawCell(t.x, t.y, state.grid[r][c], m ? m.scale : 1, m ? m.flash : 0);
    }
  }

  // 第三遍：冲击波光环（方块之上）
  for (let r = 0; r < SIZE; r++) {
    for (let c = 0; c < SIZE; c++) {
      const m = mmap[r * SIZE + c];
      if (!m) continue;
      const ct = cellCenter(r, c);
      drawRing(ct.x, ct.y, m.k);
    }
  }

  // 火花粒子
  for (const p of particles) {
    const a = 1 - p.life / p.maxLife;
    ctx.save();
    ctx.globalAlpha = Math.max(0, a);
    ctx.fillStyle = p.color;
    ctx.beginPath();
    ctx.arc(p.x, p.y, p.size, 0, Math.PI * 2);
    ctx.fill();
    ctx.restore();
  }

  // 分数上浮消散
  for (const s of scorePops) {
    const k = (now - s.t0) / SCORE_MS;
    const a = 1 - k;
    ctx.save();
    ctx.globalAlpha = Math.max(0, a);
    ctx.fillStyle = '#f59563';
    ctx.font = 'bold 18px sans-serif';
    ctx.textAlign = 'center';
    ctx.textBaseline = 'middle';
    ctx.fillText('+' + s.val, s.x, s.y - k * 26);
    ctx.restore();
  }

  // 震动开关按钮（棋盘正下方居中，仅游戏进行中显示）
  if (screen === 'play') {
    const tg = vibrateToggleRect();
    ctx.fillStyle = vibrateOn ? '#edc22e' : '#cdc1b4';
    roundRect(tg.x, tg.y, tg.w, tg.h, tg.h / 2);
    ctx.fill();
    ctx.fillStyle = vibrateOn ? '#5b4a1f' : '#776e65';
    ctx.font = 'bold 14px sans-serif';
    ctx.textAlign = 'center';
    ctx.textBaseline = 'middle';
    ctx.fillText('震动' + (vibrateOn ? '开' : '关'), W / 2, tg.y + tg.h / 2);
  }

  // 排行榜按钮（右上角，仅游戏进行中显示）

```

```
if (screen === 'play') {
  const rb = rankBtnRect();
  ctx.fillStyle = '#8f7a66';
  roundRect(rb.x, rb.y, rb.w, rb.h, rb.h / 2);
  ctx.fill();
  ctx.fillStyle = '#fff';
  ctx.font = 'bold 14px sans-serif';
  ctx.textAlign = 'center';
  ctx.textBaseline = 'middle';
  ctx.fillText('排行榜', rb.x + rb.w / 2, rb.y + rb.h / 2);
}

// 天梯入口按钮（左上角，紧挨排行榜按钮右侧，避开系统胶囊），仅游戏进行中显示
if (screen === 'play') {
  const lb = Ladder.ladderEntryBtnRect();
  ctx.fillStyle = '#8f7a66';
  roundRect(lb.x, lb.y, lb.w, lb.h, lb.h / 2);
  ctx.fill();
  ctx.fillStyle = '#fff';
  ctx.font = 'bold 14px sans-serif';
  ctx.textAlign = 'center';
  ctx.textBaseline = 'middle';
  ctx.fillText('天梯', lb.x + lb.w / 2, lb.y + lb.h / 2);
}

// 房间入口按钮（左上角，紧挨天梯按钮右侧，避开系统胶囊），仅游戏进行中显示
if (screen === 'play') {
  const rm = roomEntryBtnRect();
  ctx.fillStyle = '#8f7a66';
  roundRect(rm.x, rm.y, rm.w, rm.h, rm.h / 2);
  ctx.fill();
  ctx.fillStyle = '#fff';
  ctx.font = 'bold 14px sans-serif';
  ctx.textAlign = 'center';
  ctx.textBaseline = 'middle';
  ctx.fillText('房间', rm.x + rm.w / 2, rm.y + rm.h / 2);
}

// 游戏结束遮罩
if (state.over && screen === 'play') {
  ctx.fillStyle = 'rgba(250,248,239,0.80)';
  ctx.fillRect(boardX, boardY, boardSize, boardSize);
  ctx.textAlign = 'center';
  ctx.textBaseline = 'middle';

  // 标题
  ctx.fillStyle = '#776e65';
  ctx.font = 'bold 26px sans-serif';
  ctx.fillText('游戏结束', W / 2, H / 2 - 64);

  // 本局得分
  ctx.fillStyle = '#5b4a1f';
  ctx.font = 'bold 18px sans-serif';
  ctx.fillText('本局得分：' + state.score, W / 2, H / 2 - 30);

  // 个人排行（按优先级：加载中 → 有效排名 → 暂不可用）
  let rankLine = '排名暂不可用';
  if (rankLoading) {
    rankLine = '排名加载中...';
  } else if (rankData && Number(rankData.selfRank) > 0) {
    rankLine = '你的排名：第 ' + rankData.selfRank + ' 位';
  }
}
```

```
ctx.fillStyle = '#776e65';
ctx.font = '15px sans-serif';
ctx.fillText(rankLine, W / 2, H / 2 + 2);

// 底部「再来一局」按钮（圆角，底色 #edc22e，文字色 #5b4a1f）
const bw = 160, bh = 44;
const bx = W / 2 - bw / 2, by = H / 2 + 30;
ctx.fillStyle = '#edc22e';
roundRect(bx, by, bw, bh, bh / 2);
ctx.fill();
ctx.fillStyle = '#5b4a1f';
ctx.font = 'bold 15px sans-serif';
ctx.fillText('再来一局', W / 2, by + bh / 2);
}

// 开局引导
if (screen === 'guide') {
  ctx.fillStyle = 'rgba(0,0,0,0.62)';
  ctx.fillRect(0, 0, W, H);
  ctx.fillStyle = '#ffffff';
  ctx.textAlign = 'center';
  ctx.textBaseline = 'middle';
  ctx.font = 'bold 26px sans-serif';
  ctx.fillText('合成能量', W / 2, H / 2 - 130);
  ctx.font = '15px sans-serif';
  const lines = [
    '滑动屏幕，相同数字撞一起就合成翻倍',
    '2+2=4，4+4=8，越合越大',
    '每步随机冒出新数字，别让格子填满',
    '目标：挑战你的最高分！',
  ];
  lines.forEach((ln, i) => ctx.fillText(ln, W / 2, H / 2 - 80 + i * 26));
  const bx = W / 2 - 80, by = H / 2 + 60, bw = 160, bh = 48;
  ctx.fillStyle = '#edc22e';
  ctx.fillRect(bx, by, bw, bh);
  ctx.fillStyle = '#5b4a1f';
  ctx.font = 'bold 20px sans-serif';
  ctx.fillText('开始游戏', W / 2, by + bh / 2);
}

// 排行榜弹窗（覆盖在最上层）
if (screen === 'rank') drawRank();

// 天梯结算卡（覆盖在最上层）
if (screen === 'ladder') drawLadder();
// 天梯战绩面板（覆盖在最上层）
if (screen === 'ladderHistory') drawLadderHistory();
// 房间对战 UI（覆盖在最上层；screen==='room' 时由 Room 接管绘制）
if (screen === 'room') { Room.setStateRef(state); Room.render(ctx, L); }
}

// 天梯结算卡绘制（数据来自 ladderMatch / ladderLoading / ladderError）
function drawLadder() {
  Ladder.drawLadderCard(ctx, L, ladderMatch, {
    loading: ladderLoading,
    error: ladderError,
    selfRank: (rankData && rankData.selfRank) || 0,
    score: state.score,
  });
}

// 天梯战绩面板绘制（数据来自 ladderHist / ladderHistLoading / ladderHistError）
```

```

function drawLadderHistory() {
  const clamped = Ladder.drawLadderHistory(ctx, L, ladderHist, {
    loading: ladderHistLoading, error: ladderHistError, scroll: ladderScroll,
  });
  ladderScroll = clamped;
}

let lastT = Date.now();
function loop() {
  const now = Date.now();
  const dt = Math.min(50, now - lastT);
  lastT = now;
  updateParticles(dt);
  draw();
  if (isTT) requestAnimationFrame(loop);
}

if (isTT) {
  requestAnimationFrame(loop);
} else {
  // 非小游戏环境（例如误在 Node 下执行）给出提示，不崩溃
  console.log('[merge-energy] 这是抖音小游戏工程，请在「抖音开发者工具」中导入运行。');
}

// 测试钩子（仅供 QA 在 Node 下 require 验证；不影响抖音运行时）
if (typeof module !== 'undefined' && module.exports) {
  module.exports = {
    _t: {
      getScroll: () => rankScroll,
      setScroll: (v) => { rankScroll = v; },
      openRank,
      isTT,
      draw, // QA 钩子：在 Node 下调用一次渲染帧，用于断言「游戏结束遮罩 / 排名降级文案」实际被绘制（ctx 为 mock，无副作用）
      getScreen: () => screen,
      getRankState: () => ({ loading: rankLoading, error: rankError, data: rankData, uid: rankUid, name: rankSelfName }),
      loseGame: () => { state.over = true; if (isTT) triggerGameOver(); },
      setRankError: (v) => { rankError = v; },
      rankBtnRect,
      rankCloseRect, // QA 钩子：返回排行榜关闭 × 的屏幕矩形，供回归测试按真实生产坐标驱动 touch
      getRankReturnScreen: () => rankReturnScreen, // QA 钩子：返回打开榜单时记录的来源界面
      roomEntryBtnRect,
      openRoom,
      getRoomPhase: () => Room.getPhase(),
      getRoomState: () => Room.getState(),
      roomHandleTouch: (sx, sy, t) => Room.handleTouch(sx, sy, t),
      roomCreate: () => Room.createRoom(),
      roomJoin: (c) => Room.joinRoom(c),
      roomReport: (s, st, o) => Room.reportProgress(s, st, o),
      roomResult: (s, st, w) => Room.submitMyResult(s, st, w),
      roomExit: () => Room.exit(),
      roomRestart: () => Room.requestRestart(),
      getRoomRng: () => roomRng,
      ladderEntryBtnRect: () => Ladder.ladderEntryBtnRect(),
      openLadderHistory,
      getLadderState: () => ({
        screen,
        match: ladderMatch, loading: ladderLoading, error: ladderError,
        hist: ladderHist, histLoading: ladderHistLoading, histError: ladderHistError,
      }),
    },
  };
}

```